DSC259 Final Project: Power Outages

Scotty Rogers Jillian ONeel Hans Hanson

Table of contents

1 DSC259 Final Project: Power Outages 2
1.1 Step 1: Introduction . . . 3
1.2 Step 2: Data Cleaning and Exploratory Analysis . . . 3
1.2.1 Initial Inspection . . . 3
1.2.2 Load Data . . . 4
1.2.3 Collect Cleaning Steps . . . 10
1.2.4 Data Cleaning . . . 24
1.2.5 Univariate Analysis . . . 27
1.2.6 Bivariate Analysis . . . 32
1.2.7 Interesting Aggregates . . . 34
1.3 Step 3: Assessment of Missingness . . . 35
1.3.1 NMAR Analysis . . . 35
1.3.2 Missingness Dependency Tests . . . 36
1.3.3 Missingness Dependency: Interpretation . . . 39
1.3.4 Summary . . . 39
1.4 Step 4: Hypothesis Testing . . . 40
1.4.1 Hypothesis Test: Does Outage Duration Vary by State Within the Same NERC Region? . . . 40
1.4.2 **Step 4 Conclusions** . . . 41
1.5 Step 5: Framing a Prediction Problem . . . 42
1.6 Step 6: Baseline Model . . . 44
1.6.1 Baseline Model: Ridge Regression with Log-Transformed Target . . . 44
1.6.2 **Why Use Ridge?** . . . 44
1.6.3 Feature Classification . . . 44
1.6.4 **Model Comparison: Solving the Baseline Failures** . . . 47
1.6.5 **Solving the Baseline Feature Failures** . . . 48
1.7 Step 7: Final Model . . . 50
1.7.1 Final Model Overview . . . 50
1.7.2 Final Model Plots . . . 55
1.7.3 Final Model Results . . . 57
1.8 Step 8: Fairness Analysis . . . 57
1.8.1 Project Summary . . . 61
1.8.2 Closing Remarks . . . 62

1 DSC259 Final Project: Power Outages

Name(s): Randall "Scotty" Rogers, Jillian O'Neel, Hans Hanson

Website Link: here

```
import pandas as pd
import numpy as np
import math
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots

import statsmodels.api as sm
from statsmodels.formula.api import ols

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.linear_model import Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

# Plotly as default pandas plotting backend
pd.options.plotting.backend = 'plotly'

import os
os.makedirs('../data/docs/img/plots', exist_ok=True)

# Quarto PDF patch for Plotly static rendering
import os
os.environ.setdefault("MPLBACKEND", "Agg")
try:
    import plotly.io as pio
    pio.renderers.default = "png"
    try:
        pio.kaleido.scope.default_format = "png"
        pio.kaleido.scope.default_width = 1400
        pio.kaleido.scope.default_height = 900
        pio.kaleido.scope.default_scale = 2
    except Exception:
        pass
except Exception as exc:
    print("Plotly static renderer setup warning:", exc)

try:
    import pandas as pd
    pd.options.plotting.backend = "plotly"
except Exception:
    pass
```

1.1 Step 1: Introduction

- This project leverages the **Major Power Outage Risks in the U.S.** dataset from Purdue University's LASCI research group. This dataset records 1,534 major outage events across the continental United States from 2000 through 2016. Beyond simple timestamps, the data tracks a wide range of factors, including climate conditions, local geography, and state-level economic indicators, providing a comprehensive view of components associated with outage severity and disruption.
- **Central Question:** *Can we accurately predict an outage's impact on* `CUSTOMERS.AFFECTED` *using only the information available at its onset?*
  - **Why focus on customers affected?** This is the most direct measure of public impact. Larger outages create substantially higher risks for households, critical infrastructure, and emergency response systems.
  - **Why this matters:** Providing early, reliable estimates of outage severity allows utilities and emergency managers to allocate resources where they are needed most. Our goal is not just to build a predictive model, but also to investigate whether outage patterns shift across state lines and evaluate whether the model performs consistently across different groups.

1.2 Step 2: Data Cleaning and Exploratory Analysis

1.2.1 Initial Inspection

- **Initial Inspection: Reformat** `outage.xlsx`
  - After downloading save the dataset as a *.csv* file
  - Using the following command on the downloaded dataset reveals a mess:

```
head data/outage.csv
```

```
Major power outage events in the continental
    U.S.,,,
```

```
,1,2011,7,Minnesota,MN,MRO,East North Central,-0.3,normal,"Friday, July 1,
    2011",5:00:00 PM,"Sunday, July 3, 2011",8:00:00 PM,severe
    weather,NA,NA,3060,NA,70000,11.6,9.18,6.81,9.28,2332915,2114774,2113291,6562520,35.5490726123501
,2,2014,5,Minnesota,MN,MRO,East North Central,-0.1,normal,"Sunday, May 11,
    2014",6:38:00 PM,"Sunday, May 11, 2014",6:39:00 PM,intentional
    attack,vandalism,NA,1,NA,NA,12.12,9.71,6.49,9.28,1586986,1807756,1887927,5284231,30.032487224725
,3,2010,10,Minnesota,MN,MRO,East North Central,-1.5,cold,"Tuesday, October 26,
    2010",8:00:00 PM,"Thursday, October 28, 2010",10:00:00 PM,severe weather,heavy
    wind,NA,3000,NA,70000,10.87,8.19,6.07,8.15,1467293,1801683,1951295,5222116,28.0976715185951,34.5
```

- **Initial Inspection: Quick Clean**
  - We can perform a *quick clean*, to reformat the dataset with the following commands:

```
head -6 data/outage.csv | tail -1 | cut -d ',' -f 2- > data/outage_clean.csv
```

```
tail -n +8 data/outage.csv | cut -d ',' -f 2- >> data/outage_clean.csv
```

1.2.2 Load Data

- Here we will load the data as a pandas DataFrame then do the following:
  - 1. Read the reformatted power outage data into a DataFrame using pandas `.read_csv()`
  - 2. Check the number of columns, rows, and total observations in the power outage dataset
  - 3. Use the `.info()` method to get concise summary of the power outage dataset
  - 4. Check null or na count for each column
  - 5. Use the `.describe()` method to get the summary statistics for all numeric columns
  - 6. Drop any duplicate rows in the dataset

```
# 1. # Read the reformatted power outage data into a DataFrame using pandas
    `.read_csv()`
data = "../data/outage_clean.csv"
df = pd.read_csv(data)

# 2. Check the number of columns, rows, and total observations in the dataset
label_w = 30
num_w = 8
orginal_col_count = df.shape[1]
orginal_row_count = df.shape[0]
print(f"{'Columns in Outage Data:':>{label_w}} {orginal_col_count:>{num_w}}")
print(f"{'Rows in Outage Data:':>{label_w}} {orginal_row_count:>{num_w}}")
print(f"{'Total Elements in Outage Data:':>{label_w}} {df.size:>{num_w}}")

# 3. Use the `.info()` method to get concise summary of the power outage dataset
print("\nDataset Info:")
df.info()

# 4. Check null or na count for each column
print("\nMissing Values per Column:")
```

```python
print(df.isna().sum()[df.isna().sum() > 0])

# 5. Use the `.describe()` method to get the summary statistics for all numeric
    columns
print("\nSummary Statistics for Numeric Columns:")
display(df.describe())

# 6. Drop any duplicate rows in the dataset
cols_to_compare = [c for c in df.columns if c != 'OBS']
df = df.drop_duplicates(subset=cols_to_compare)
print(f"\nRows removed: {orginal_row_count - len(df)}")
print(f"Final row count after dropping true duplicates: {len(df)}")
```

```
       Columns in Outage Data:       56
          Rows in Outage Data:     1534
Total Elements in Outage Data:    85904

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1534 entries, 0 to 1533
Data columns (total 56 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   OBS                      1534 non-null   int64
 1   YEAR                     1534 non-null   int64
 2   MONTH                    1525 non-null   float64
 3   U.S._STATE               1534 non-null   object
 4   POSTAL.CODE              1534 non-null   object
 5   NERC.REGION              1534 non-null   object
 6   CLIMATE.REGION           1528 non-null   object
 7   ANOMALY.LEVEL            1525 non-null   float64
 8   CLIMATE.CATEGORY         1525 non-null   object
 9   OUTAGE.START.DATE        1525 non-null   object
 10  OUTAGE.START.TIME        1525 non-null   object
 11  OUTAGE.RESTORATION.DATE  1476 non-null   object
 12  OUTAGE.RESTORATION.TIME  1476 non-null   object
 13  CAUSE.CATEGORY           1534 non-null   object
 14  CAUSE.CATEGORY.DETAIL    1063 non-null   object
 15  HURRICANE.NAMES          72 non-null     object
 16  OUTAGE.DURATION          1476 non-null   float64
 17  DEMAND.LOSS.MW           829 non-null    float64
 18  CUSTOMERS.AFFECTED       1091 non-null   float64
 19  RES.PRICE                1512 non-null   float64
 20  COM.PRICE                1512 non-null   float64
 21  IND.PRICE                1512 non-null   float64
 22  TOTAL.PRICE              1512 non-null   float64
 23  RES.SALES                1512 non-null   float64
 24  COM.SALES                1512 non-null   float64
 25  IND.SALES                1512 non-null   float64
 26  TOTAL.SALES              1512 non-null   float64
 27  RES.PERCEN               1512 non-null   float64
 28  COM.PERCEN               1512 non-null   float64
 29  IND.PERCEN               1512 non-null   float64
 30  RES.CUSTOMERS            1534 non-null   int64
```

```
 31  COM.CUSTOMERS            1534 non-null   int64
 32  IND.CUSTOMERS            1534 non-null   int64
 33  TOTAL.CUSTOMERS          1534 non-null   int64
 34  RES.CUST.PCT             1534 non-null   float64
 35  COM.CUST.PCT             1534 non-null   float64
 36  IND.CUST.PCT             1534 non-null   float64
 37  PC.REALGSP.STATE         1534 non-null   int64
 38  PC.REALGSP.USA           1534 non-null   int64
 39  PC.REALGSP.REL           1534 non-null   float64
 40  PC.REALGSP.CHANGE        1534 non-null   float64
 41  UTIL.REALGSP             1534 non-null   int64
 42  TOTAL.REALGSP            1534 non-null   int64
 43  UTIL.CONTRI              1534 non-null   float64
 44  PI.UTIL.OFUSA            1534 non-null   float64
 45  POPULATION               1534 non-null   int64
 46  POPPCT_URBAN             1534 non-null   float64
 47  POPPCT_UC                1534 non-null   float64
 48  POPDEN_URBAN             1534 non-null   float64
 49  POPDEN_UC                1524 non-null   float64
 50  POPDEN_RURAL             1524 non-null   float64
 51  AREAPCT_URBAN            1534 non-null   float64
 52  AREAPCT_UC               1534 non-null   float64
 53  PCT_LAND                 1534 non-null   float64
 54  PCT_WATER_TOT            1534 non-null   float64
 55  PCT_WATER_INLAND         1534 non-null   float64
dtypes: float64(33), int64(11), object(12)
memory usage: 671.3+ KB

Missing Values per Column:
MONTH                         9
CLIMATE.REGION                6
ANOMALY.LEVEL                 9
CLIMATE.CATEGORY              9
OUTAGE.START.DATE             9
OUTAGE.START.TIME             9
OUTAGE.RESTORATION.DATE      58
OUTAGE.RESTORATION.TIME      58
CAUSE.CATEGORY.DETAIL       471
HURRICANE.NAMES            1462
OUTAGE.DURATION              58
DEMAND.LOSS.MW              705
CUSTOMERS.AFFECTED          443
RES.PRICE                    22
COM.PRICE                    22
IND.PRICE                    22
TOTAL.PRICE                  22
RES.SALES                    22
COM.SALES                    22
IND.SALES                    22
TOTAL.SALES                  22
RES.PERCEN                   22
COM.PERCEN                   22
IND.PERCEN                   22
POPDEN_UC                    10
```

```
POPDEN_RURAL                 10
dtype: int64

Summary Statistics for Numeric Columns:

Rows removed: 8
Final row count after dropping true duplicates: 1526
```

	OBS	YEAR	MONTH	ANOMALY.LEVEL	OUTAGE.DURATION	DEMAND.L
count	1534.000000	1534.000000	1525.000000	1525.000000	1476.000000	829.000000
mean	767.500000	2010.119296	6.234754	-0.096852	2625.398374	536.287093
std	442.971971	3.822306	3.254510	0.739957	5942.483307	2196.450393
min	1.000000	2000.000000	1.000000	-1.600000	0.000000	0.000000
25%	384.250000	2008.000000	4.000000	-0.500000	102.250000	3.000000
50%	767.500000	2011.000000	6.000000	-0.300000	701.000000	168.000000
75%	1150.750000	2013.000000	9.000000	0.300000	2880.000000	400.000000
max	1534.000000	2016.000000	12.000000	2.300000	108653.000000	41788.000000

Load Data: Comments

- **Structure and Size:** The dataset contains **1,526 unique records** (after removing 8 duplicates) across **56 columns**, consisting of a mix of numerical (float64/int64) and categorical (object) data.
- **Missing Data Challenges:** Several critical columns have significant gaps, most notably **HURRICANE.NAMES** (95% missing), **DEMAND.LOSS.MW** (~46% missing), and **CAUSE.CATEGORY.DETAIL** (~31% missing).
- **Feature Diversity:** The data is highly multidimensional, merging power outage specifics (start/restoration times, cause) with geographic climate regions, economic indicators (GSP, utility contributions), and detailed demographic/population density metrics.
  - For this reason we will do the following:
    * 1. Define an `inspect_dataframe` function that can be reused for for each of the categories in the table below.
    * 2. Working with 56 columns is difficult to manage, so we will define a column categories dictonary (`column_categories`) that organizes the data into the following:

Category	Variables
Identifier	`OBS`
Temporal	`YEAR`, `MONTH`, `OUTAGE.START.DATE`, `OUTAGE.START.TIME`, `OUTAGE.RESTORATION.DATE`, `OUTAGE.RESTORATION.TIME`, `OUTAGE.DURATION`
Spatial and Climate	`U.S._STATE`, `POSTAL.CODE`, `NERC.REGION`, `CLIMATE.REGION`, `CLIMATE.CATEGORY`, `ANOMALY.LEVEL`

Category	Variables
Outage Impact and Cause	CAUSE.CATEGORY, CAUSE.CATEGORY.DETAIL, HURRICANE.NAMES, DEMAND.LOSS.MW, CUSTOMERS.AFFECTED
Electricity Economics	RES.PRICE, COM.PRICE, IND.PRICE, TOTAL.PRICE, RES.SALES, COM.SALES, IND.SALES, TOTAL.SALES, RES.PERCEN, COM.PERCEN, IND.PERCEN, RES.CUSTOMERS, COM.CUSTOMERS, IND.CUSTOMERS, TOTAL.CUSTOMERS, RES.CUST.PCT, COM.CUST.PCT, IND.CUST.PCT
State Economics	PC.REALGSP.STATE, PC.REALGSP.USA, PC.REALGSP.REL, PC.REALGSP.CHANGE, UTIL.REALGSP, TOTAL.REALGSP, UTIL.CONTRI, PI.UTIL.OFUSA
Demographics and Land Use	POPULATION, POPPCT_URBAN, POPPCT_UC, POPDEN_URBAN, POPDEN_UC, POPDEN_RURAL, AREAPCT_URBAN, AREAPCT_UC, PCT_LAND, PCT_WATER_TOT, PCT_WATER_INLAND

```python
# 1. Define an `inspect_dataframe` function
def inspect_dataframe(df, column_category_key, print_value_counts=False):
    """
    Inspects a DataFrame and prints the following:
        - head() results
        - info() results
        - summary statistics for numeric columns
        - NaN counts
        - value counts for categorical columns (if print_value_counts is True)

    Parameters:
        df: pd.DataFrame
        column_category_key: str
        print_value_counts: bool

    Returns:
        None
    """
    # Print the first 5 rows of the DataFrame
    print("First 5 rows of the DataFrame:")
    display(df[column_categories[column_category_key]].head())

    # Print concise summary of the DataFrame subsetted to the specified column
    category
    print("\nDataFrame Info:")
    display(df[column_categories[column_category_key]].info())

    # Print summary statistics for numeric columns
    print("\nSummary Statistics for Numeric Columns:")
    display(df[column_categories[column_category_key]].describe())
```

```python
    # Check for missing values
    print("\nMissing Values (NaN) per Column:")
    # Iterate through your pre-defined category keys
    for col_i in column_categories[column_category_key]:
        null_count = df[col_i].isnull().sum()
        print(f"  {col_i}: {null_count}")

    # Print value counts for categorical columns if print_value_counts is True
    if print_value_counts:
        print("\nValue Counts for Categorical Columns:")
        for col_i in column_categories[column_category_key]:
            if df[col_i].dtype == 'object' or df[col_i].dtype.name == 'category':
                print(f"\nValue counts for {col_i}:")
                display(df[col_i].value_counts())

# 2. Define a column categories dictonary
column_categories = {
    "identifier": [
        "OBS"
    ],
    "temporal": [
        "YEAR", "MONTH",
        "OUTAGE.START.DATE", "OUTAGE.START.TIME",
        "OUTAGE.RESTORATION.DATE", "OUTAGE.RESTORATION.TIME",
        "OUTAGE.DURATION"
    ],
    "spatial_and_climate": [
        "U.S._STATE", "POSTAL.CODE", "NERC.REGION",
        "CLIMATE.REGION", "CLIMATE.CATEGORY", "ANOMALY.LEVEL"
    ],
    "outage_impact_and_cause": [
        "CAUSE.CATEGORY", "CAUSE.CATEGORY.DETAIL", "HURRICANE.NAMES",
        "DEMAND.LOSS.MW", "CUSTOMERS.AFFECTED"
    ],
    "electricity_economics": [
        "RES.PRICE", "COM.PRICE", "IND.PRICE", "TOTAL.PRICE",
        "RES.SALES", "COM.SALES", "IND.SALES", "TOTAL.SALES",
        "RES.PERCEN", "COM.PERCEN", "IND.PERCEN",
        "RES.CUSTOMERS", "COM.CUSTOMERS", "IND.CUSTOMERS", "TOTAL.CUSTOMERS",
        "RES.CUST.PCT", "COM.CUST.PCT", "IND.CUST.PCT"
    ],
    "state_economics": [
        "PC.REALGSP.STATE", "PC.REALGSP.USA", "PC.REALGSP.REL",
        "PC.REALGSP.CHANGE", "UTIL.REALGSP", "TOTAL.REALGSP",
        "UTIL.CONTRI", "PI.UTIL.OFUSA"
    ],
    "demographics_and_land_use": [
        "POPULATION", "POPPCT_URBAN", "POPPCT_UC",
        "POPDEN_URBAN", "POPDEN_UC", "POPDEN_RURAL",
        "AREAPCT_URBAN", "AREAPCT_UC",
        "PCT_LAND", "PCT_WATER_TOT", "PCT_WATER_INLAND"
    ]
}
```

1.2.3 Collect Cleaning Steps

- The `column_categories` dictionary serves as the framework for identifying necessary data cleaning procedures.

1.2.3.1 Inspecting the `identifier` key

```
inspect_dataframe(df, "identifier")
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   OBS     1526 non-null   int64
dtypes: int64(1)
memory usage: 23.8 KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  OBS: 0
```

	OBS
0	1
1	2
2	3
3	4
4	5

```
None
```

	OBS
count	1526.000000
mean	767.485583
std	443.159154
min	1.000000
25%	384.250000
50%	766.500000
75%	1151.750000
max	1534.000000

1.2.3.2 Inspecting the `identifier` key: Comments

- The OBS column functions as a row index; however, this is redundant as the data is more efficiently managed via the native df.index.
- Note: While OBS utilizes 1-based indexing, df.index follows standard 0-based Python indexing.
- **Step 1 Data Cleaning**: Drop the OBS column to streamline the dataset.

1.2.3.3 Inspecting the `temporal` key

```
inspect_dataframe(df, "temporal")
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 7 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   YEAR                     1526 non-null   int64
 1   MONTH                    1517 non-null   float64
 2   OUTAGE.START.DATE        1517 non-null   object
 3   OUTAGE.START.TIME        1517 non-null   object
 4   OUTAGE.RESTORATION.DATE  1468 non-null   object
 5   OUTAGE.RESTORATION.TIME  1468 non-null   object
 6   OUTAGE.DURATION          1468 non-null   float64
dtypes: float64(2), int64(1), object(4)
memory usage: 95.4+ KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  YEAR: 0
  MONTH: 9
  OUTAGE.START.DATE: 9
  OUTAGE.START.TIME: 9
  OUTAGE.RESTORATION.DATE: 58
  OUTAGE.RESTORATION.TIME: 58
  OUTAGE.DURATION: 58
```

	YEAR	MONTH	OUTAGE.START.DATE	OUTAGE.START.TIME	OUTAGE.RESTORATION.DAT
0	2011	7.0	Friday, July 1, 2011	5:00:00 PM	Sunday, July 3, 2011
1	2014	5.0	Sunday, May 11, 2014	6:38:00 PM	Sunday, May 11, 2014
2	2010	10.0	Tuesday, October 26, 2010	8:00:00 PM	Thursday, October 28, 2010
3	2012	6.0	Tuesday, June 19, 2012	4:30:00 AM	Wednesday, June 20, 2012
4	2015	7.0	Saturday, July 18, 2015	2:00:00 AM	Sunday, July 19, 2015

```
None
```

	YEAR	MONTH	OUTAGE.DURATION
count	1526.00000	1517.000000	1468.000000
mean	2010.12713	6.232696	2582.575613
std	3.82068	3.255892	5813.373985
min	2000.00000	1.000000	0.000000
25%	2008.00000	4.000000	100.000000
50%	2011.00000	6.000000	691.000000
75%	2013.00000	9.000000	2880.000000
max	2016.00000	12.000000	108653.000000

1.2.3.4 Inspecting the `temporal` key: Comments

- To streamline the dataset and prepare for duration-based analysis, we will perform the following operations:
  - **Step 2 Data Cleaning**: Combine `OUTAGE.START.DATE` and `OUTAGE.START.TIME` into a single column `outage_start_dt`.
  - **Step 3 Data Cleaning**: Combine `OUTAGE.RESTORATION.DATE` and `OUTAGE.RESTORATION.TIME` into a single column `outage_restoration_dt`.
  - **Step 4 Data Cleaning**: Drop the original date and time columns to eliminate redundancy.
  - **Step 5 Data Cleaning**: Drop `NaN` values in `outage_start_dt` and `outage_restoration_dt` to ensure data integrity for future calculations.

1.2.3.5 Inspecting the `spatial_and_climate` key

```
inspect_dataframe(df, "spatial_and_climate", print_value_counts=True)
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 6 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   U.S._STATE        1526 non-null   object
 1   POSTAL.CODE       1526 non-null   object
 2   NERC.REGION       1526 non-null   object
 3   CLIMATE.REGION    1520 non-null   object
 4   CLIMATE.CATEGORY  1517 non-null   object
 5   ANOMALY.LEVEL     1517 non-null   float64
dtypes: float64(1), object(5)
```

```
memory usage: 83.5+ KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  U.S._STATE: 0
  POSTAL.CODE: 0
  NERC.REGION: 0
  CLIMATE.REGION: 6
  CLIMATE.CATEGORY: 9
  ANOMALY.LEVEL: 9

Value Counts for Categorical Columns:

Value counts for U.S._STATE:

Value counts for POSTAL.CODE:

Value counts for NERC.REGION:

Value counts for CLIMATE.REGION:

Value counts for CLIMATE.CATEGORY:
```

	U.S._STATE	POSTAL.CODE	NERC.REGION	CLIMATE.REGION	CLIMATE.CATEGORY	ANO
0	Minnesota	MN	MRO	East North Central	normal	-0.3
1	Minnesota	MN	MRO	East North Central	normal	-0.1
2	Minnesota	MN	MRO	East North Central	cold	-1.5
3	Minnesota	MN	MRO	East North Central	normal	-0.1
4	Minnesota	MN	MRO	East North Central	warm	1.2

```
None
```

	ANOMALY.LEVEL
count	1517.000000
mean	-0.095254
std	0.740207
min	-1.600000
25%	-0.500000
50%	-0.300000
75%	0.300000
max	2.300000

```
U.S._STATE
California              209
Texas                   126
Washington               97
```

```
Michigan                 95
New York                 71
Maryland                 58
Pennsylvania             57
Illinois                 46
Ohio                     43
Indiana                  43
Florida                  43
Delaware                 41
Utah                     41
North Carolina           40
Louisiana                38
Virginia                 37
New Jersey               35
Tennessee                33
Arizona                  27
Oregon                   26
Arkansas                 25
Oklahoma                 24
Wisconsin                20
Maine                    19
Massachusetts            18
Connecticut              18
Georgia                  17
Missouri                 17
Minnesota                15
Colorado                 15
New Hampshire            14
Kentucky                 13
District of Columbia     10
Idaho                     9
Kansas                    9
Vermont                   9
New Mexico                8
South Carolina            8
Iowa                      8
Nevada                    7
Alabama                   6
Wyoming                   6
Hawaii                    5
Mississippi               4
West Virginia             4
Nebraska                  4
Montana                   3
North Dakota              2
South Dakota              2
Alaska                    1
Name: count, dtype: int64
```

```
POSTAL.CODE
CA    209
TX    126
WA     97
MI     95
```

```
NY     71
MD     58
PA     57
IL     46
OH     43
IN     43
FL     43
DE     41
UT     41
NC     40
LA     38
VA     37
NJ     35
TN     33
AZ     27
OR     26
AR     25
OK     24
WI     20
ME     19
MA     18
CT     18
GA     17
MO     17
MN     15
CO     15
NH     14
KY     13
DC     10
ID      9
KS      9
VT      9
NM      8
SC      8
IA      8
NV      7
AL      6
WY      6
HI      5
MS      4
WV      4
NE      4
MT      3
ND      2
SD      2
AK      1
Name: count, dtype: int64
```

```
NERC.REGION
WECC          449
RFC           419
SERC          202
NPCC          150
TRE           110
```

```
SPP            67
MRO            46
FRCC           42
ECAR           34
HECO            3
FRCC, SERC      1
HI              1
PR              1
ASCC            1
Name: count, dtype: int64
```

```
CLIMATE.REGION
Northeast             350
South                 226
West                  216
Central               199
Southeast             151
East North Central    138
Northwest             132
Southwest              91
West North Central     17
Name: count, dtype: int64
```

```
CLIMATE.CATEGORY
normal    741
cold      469
warm      307
Name: count, dtype: int64
```

1.2.3.6 Inspecting the `spatial_and_climate` key: Comments

- The spatial and climate subset contains 1,526 records dominated by California (209) and the Northeast region (350), showing high data density in major coastal states and relatively complete records, with minimal missingness (less than 1%) in climate-specific attributes.
- To address the missingness in the climate-specific attributes we will do the following:
  - **Step 6 Data Cleaning**: Fill `CLIMATE.REGION` null values with the value from U.S._STATE + " Region"

1.2.3.7 Inspecting the `outage_impact_and_cause` key

```
inspect_dataframe(df, "outage_impact_and_cause", print_value_counts=True)
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 5 columns):
 #   Column                 Non-Null Count  Dtype
```

```
 ---  ------                  --------------  -----
  0   CAUSE.CATEGORY          1526 non-null   object
  1   CAUSE.CATEGORY.DETAIL   1059 non-null   object
  2   HURRICANE.NAMES         72 non-null     object
  3   DEMAND.LOSS.MW          826 non-null    float64
  4   CUSTOMERS.AFFECTED      1089 non-null   float64
dtypes: float64(2), object(3)
memory usage: 71.5+ KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  CAUSE.CATEGORY: 0
  CAUSE.CATEGORY.DETAIL: 467
  HURRICANE.NAMES: 1454
  DEMAND.LOSS.MW: 700
  CUSTOMERS.AFFECTED: 437

Value Counts for Categorical Columns:

Value counts for CAUSE.CATEGORY:

Value counts for CAUSE.CATEGORY.DETAIL:

Value counts for HURRICANE.NAMES:
```

	CAUSE.CATEGORY	CAUSE.CATEGORY.DETAIL	HURRICANE.NAMES	DEMAND.LOSS.MW	C
0	severe weather	NaN	NaN	NaN	70
1	intentional attack	vandalism	NaN	NaN	N
2	severe weather	heavy wind	NaN	NaN	70
3	severe weather	thunderstorm	NaN	NaN	68
4	severe weather	NaN	NaN	250.0	25

```
None
```

	DEMAND.LOSS.MW	CUSTOMERS.AFFECTED
count	826.000000	1.089000e+03
mean	537.602906	1.437197e+05
std	2200.281432	2.871840e+05
min	0.000000	0.000000e+00
25%	3.000000	9.700000e+03
50%	168.000000	7.079300e+04
75%	399.750000	1.500000e+05
max	41788.000000	3.241437e+06

```
CAUSE.CATEGORY
severe weather                  760
```

```
intentional attack                418
system operability disruption    127
public appeal                     65
equipment failure                 60
fuel supply emergency             50
islanding                         46
Name: count, dtype: int64
```

```
CAUSE.CATEGORY.DETAIL
vandalism                    335
thunderstorm                 178
winter storm                 101
hurricanes                    74
heavy wind                    61
storm                         41
sabotage                      32
wildfire                      24
winter                        22
transmission interruption     19
generator trip                16
uncontrolled loss             14
snow/ice                      13
wind/rain                     12
heatwave                      10
 Coal                          9
Coal                           8
tornadoes                      8
 Natural Gas                   7
wind storm                     6
transmission trip              6
Hydro                          5
flooding                       4
breaker trip                   4
earthquake                     4
lightning                      3
relaying malfunction           3
hailstorm                      3
wind                           3
transmission                   3
transformer outage             3
suspicious activity            3
switching                      2
substation                     2
public appeal                  2
line fault                     2
distribution interruption      2
computer hardware              1
snow/ice storm                 1
shed load                      1
majorsystem interruption       1
voltage reduction              1
 Hydro                         1
feeder shutdown                1
plant trip                     1
```

```
100 MW loadshed                1
fog                            1
HVSubstation interruption      1
cables                         1
Petroleum                      1
thunderstorm; islanding        1
failure                        1
Name: count, dtype: int64
```

```
HURRICANE.NAMES
Sandy        17
Ike           9
Frances       5
Jeanne        5
Rita          4
Isabel        4
Charley       4
Isaac         4
Wilma         3
Ernesto       3
Fay           2
Gustav        2
Humberto      1
Claudette     1
Dolly         1
Cindy         1
Katrina       1
Hanna         1
Ivan          1
Ophelia       1
Gaston        1
Lily          1
Name: count, dtype: int64
```

1.2.3.8 Inspecting the `outage_impact_and_cause` key: Comments

- The outage cause and impact subset highlights that severe weather (760) and intentional attacks (418) are the primary drivers of disruptions, though the data faces significant sparsity with nearly 46% of demand loss and 95% of hurricane names missing.
- **Step 7 Data Cleaning**: Fill NaN values in with the parent CAUSE.CATEGORY.DETAIL + unknown

```
df['CAUSE.CATEGORY.DETAIL'] = df['CAUSE.CATEGORY.DETAIL'].fillna(
    df['CAUSE.CATEGORY'] + " unknown")
```

- **Step 8 Data Cleaning**: Create a binary classifier column based off of HURRICANE.NAMES NaNs

```
df['is_hurricane'] = df['HURRICANE.NAMES'].notnull()
```

1.2.3.9 Inspecting the `electricity_economics` key

```
inspect_dataframe(df, "electricity_economics")
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 18 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   RES.PRICE        1504 non-null   float64
 1   COM.PRICE        1504 non-null   float64
 2   IND.PRICE        1504 non-null   float64
 3   TOTAL.PRICE      1504 non-null   float64
 4   RES.SALES        1504 non-null   float64
 5   COM.SALES        1504 non-null   float64
 6   IND.SALES        1504 non-null   float64
 7   TOTAL.SALES      1504 non-null   float64
 8   RES.PERCEN       1504 non-null   float64
 9   COM.PERCEN       1504 non-null   float64
 10  IND.PERCEN       1504 non-null   float64
 11  RES.CUSTOMERS    1526 non-null   int64
 12  COM.CUSTOMERS    1526 non-null   int64
 13  IND.CUSTOMERS    1526 non-null   int64
 14  TOTAL.CUSTOMERS  1526 non-null   int64
 15  RES.CUST.PCT     1526 non-null   float64
 16  COM.CUST.PCT     1526 non-null   float64
 17  IND.CUST.PCT     1526 non-null   float64
dtypes: float64(14), int64(4)
memory usage: 226.5 KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  RES.PRICE: 22
  COM.PRICE: 22
  IND.PRICE: 22
  TOTAL.PRICE: 22
  RES.SALES: 22
  COM.SALES: 22
  IND.SALES: 22
  TOTAL.SALES: 22
  RES.PERCEN: 22
  COM.PERCEN: 22
  IND.PERCEN: 22
  RES.CUSTOMERS: 0
  COM.CUSTOMERS: 0
  IND.CUSTOMERS: 0
  TOTAL.CUSTOMERS: 0
  RES.CUST.PCT: 0
  COM.CUST.PCT: 0
  IND.CUST.PCT: 0
```

	RES.PRICE	COM.PRICE	IND.PRICE	TOTAL.PRICE	RES.SALES	COM.SALES	IND.SALES
0	11.60	9.18	6.81	9.28	2332915.0	2114774.0	2113291.0
1	12.12	9.71	6.49	9.28	1586986.0	1807756.0	1887927.0
2	10.87	8.19	6.07	8.15	1467293.0	1801683.0	1951295.0
3	11.79	9.25	6.71	9.19	1851519.0	1941174.0	1993026.0
4	13.07	10.16	7.74	10.43	2028875.0	2161612.0	1777937.0

```
None
```

	RES.PRICE	COM.PRICE	IND.PRICE	TOTAL.PRICE	RES.SALES	COM.SALES	IND.SAI
count	1504.000000	1504.000000	1504.000000	1504.000000	1.504000e+03	1.504000e+03	1.504000
mean	11.977926	10.140399	7.345691	10.123464	4.328758e+06	4.418612e+06	2.795243
std	3.090705	2.825086	2.476564	2.908094	3.376205e+06	3.495725e+06	2.214025
min	5.650000	4.700000	3.200000	4.700000	1.444170e+05	1.525170e+05	1.552100
25%	9.570000	8.020000	5.687500	7.950000	2.035200e+06	1.909914e+06	1.183986
50%	11.500000	9.480000	6.720000	9.430000	3.435784e+06	3.182069e+06	2.294875
75%	13.930000	11.350000	8.595000	11.762500	6.005798e+06	6.999389e+06	4.010162
max	34.580000	32.140000	27.850000	31.290000	1.862066e+07	1.404697e+07	9.588951

1.2.3.10 Inspecting the `electricity_economics` key: Comments

- Analysis of pricing, sales, and customer distributions gives us the 9th data cleaning step:
  - **Step 9 Data Cleaning:** Drop rows with missing values in the economics columns and start with the `TOTAL.PRICE` column.

1.2.3.11 Inspecting the `state_economics` key

```
inspect_dataframe(df, "state_economics")
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   PC.REALGSP.STATE   1526 non-null   int64
 1   PC.REALGSP.USA     1526 non-null   int64
 2   PC.REALGSP.REL     1526 non-null   float64
 3   PC.REALGSP.CHANGE  1526 non-null   float64
 4   UTIL.REALGSP       1526 non-null   int64
 5   TOTAL.REALGSP      1526 non-null   int64
 6   UTIL.CONTRI        1526 non-null   float64
 7   PI.UTIL.OFUSA      1526 non-null   float64
dtypes: float64(4), int64(4)
```

```
memory usage: 107.3 KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  PC.REALGSP.STATE: 0
  PC.REALGSP.USA: 0
  PC.REALGSP.REL: 0
  PC.REALGSP.CHANGE: 0
  UTIL.REALGSP: 0
  TOTAL.REALGSP: 0
  UTIL.CONTRI: 0
  PI.UTIL.OFUSA: 0
```

	PC.REALGSP.STATE	PC.REALGSP.USA	PC.REALGSP.REL	PC.REALGSP.CHANGE	UTIL.REA
0	51268	47586	1.077376	1.6	4802
1	53499	49091	1.089792	1.9	5226
2	50447	47287	1.066826	2.7	4571
3	51598	48156	1.071476	0.6	5364
4	54431	49844	1.092027	1.7	4873

```
None
```

	PC.REALGSP.STATE	PC.REALGSP.USA	PC.REALGSP.REL	PC.REALGSP.CHANGE	UTIL.I
count	1526.000000	1526.000000	1526.000000	1526.000000	1526.00
mean	49408.944954	48080.770642	1.027735	0.721298	11309.9
std	11710.845897	1202.646067	0.243969	2.132158	9871.24
min	31111.000000	44719.000000	0.639876	-9.100000	606.000
25%	43056.000000	47586.000000	0.893561	-0.400000	3725.00
50%	48417.000000	48156.000000	1.021204	1.000000	6971.00
75%	53622.000000	48909.000000	1.110032	1.900000	19081.0
max	168377.000000	50660.000000	3.560746	9.800000	31256.0

1.2.3.12 Inspecting the `state_economics` key: Comments

- A comprehensive look at Gross State Product (GSP) and utility industry contributions:
  - Note: No data cleaning required since there are 0 missing values across all columns.

1.2.3.13 Inspecting the `demographics_and_land_use` key

```
inspect_dataframe(df, "demographics_and_land_use")
```

```
First 5 rows of the DataFrame:

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 1526 entries, 0 to 1533
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   POPULATION        1526 non-null   int64
 1   POPPCT_URBAN      1526 non-null   float64
 2   POPPCT_UC         1526 non-null   float64
 3   POPDEN_URBAN      1526 non-null   float64
 4   POPDEN_UC         1516 non-null   float64
 5   POPDEN_RURAL      1516 non-null   float64
 6   AREAPCT_URBAN     1526 non-null   float64
 7   AREAPCT_UC        1526 non-null   float64
 8   PCT_LAND          1526 non-null   float64
 9   PCT_WATER_TOT     1526 non-null   float64
 10  PCT_WATER_INLAND  1526 non-null   float64
dtypes: float64(10), int64(1)
memory usage: 143.1 KB

Summary Statistics for Numeric Columns:

Missing Values (NaN) per Column:
  POPULATION: 0
  POPPCT_URBAN: 0
  POPPCT_UC: 0
  POPDEN_URBAN: 0
  POPDEN_UC: 10
  POPDEN_RURAL: 10
  AREAPCT_URBAN: 0
  AREAPCT_UC: 0
  PCT_LAND: 0
  PCT_WATER_TOT: 0
  PCT_WATER_INLAND: 0
```

	POPULATION	POPPCT_URBAN	POPPCT_UC	POPDEN_URBAN	POPDEN_UC	POPDEN_
0	5348119	73.27	15.28	2279.0	1700.5	18.2
1	5457125	73.27	15.28	2279.0	1700.5	18.2
2	5310903	73.27	15.28	2279.0	1700.5	18.2
3	5380443	73.27	15.28	2279.0	1700.5	18.2
4	5489594	73.27	15.28	2279.0	1700.5	18.2

```
None
```

	POPULATION	POPPCT_URBAN	POPPCT_UC	POPDEN_URBAN	POPDEN_UC	POPDE
count	1.526000e+03	1526.000000	1526.000000	1526.000000	1516.000000	1516.00
mean	1.318456e+07	80.956094	9.551094	2595.444102	1558.723417	39.5366
std	1.155458e+07	11.908394	5.247582	1084.204854	312.990285	30.9402
min	5.598510e+05	38.660000	0.000000	1232.600000	988.700000	0.40000
25%	5.310903e+06	74.570000	5.220000	2016.300000	1307.800000	15.2000
50%	8.769252e+06	84.050000	9.080000	2380.000000	1528.600000	29.5000
75%	1.940292e+07	89.810000	12.020000	2851.200000	1732.200000	54.6000
max	3.929648e+07	100.000000	40.250000	9856.500000	2220.700000	142.300

1.2.3.14 Inspecting the `demographics_and_land_use` key: Comments

- **Step 10 Data Cleaning**: Fill the missing values in `POPDEN_UC` and `POPDEN_RURAL` with 0, specifically to account for the District of Columbia being entirely urban.

1.2.4 Data Cleaning

- The following steps are from inspecting each of the keys in the `column_categories` will address the missingness previously identified:
  - **Step 1**: Drop the `OBS` column since it's just a 1-indexed version of the dataframe index.
  - **Step 2**: Merge `OUTAGE.START.DATE` and `OUTAGE.START.TIME` into `outage_start_dt`.
  - **Step 3**: Merge `OUTAGE.RESTORATION.DATE` and `OUTAGE.RESTORATION.TIME` into `outage_restoration_dt`.
  - **Step 4**: Remove the four original date and time source columns.
  - **Step 5**: Drop rows with `NaN` values in our new timestamp columns to ensure we can calculate durations.
  - **Step 6**: Fill missing `CLIMATE.REGION` values based on their `U.S._STATE`.
  - **Step 7**: Fill `NaN` in `CAUSE.CATEGORY.DETAIL` by using the parent category + "unknown".
  - **Step 8**: Create an `is_hurricane` binary column based on the presence of `HURRICANE.NAMES`.
  - **Step 9**: Drop `NaN` rows in the economics columns and remove the redundant `TOTAL.PRICE` column.
  - **Step 10**: Fill `NaN` with 0 for `POPDEN_UC` and `POPDEN_RURAL` to account for 100% urban areas like DC.

```
# 1. Drop 'OBS' column
df = df.drop(columns=['OBS'])

# Define datetime formatting
dt_format = "%A, %B %d, %Y %I:%M:%S %p"
```

```
# 2. Combine `OUTAGE.START.DATE` and `OUTAGE.START.TIME` into a single column
    `outage_start_dt`
df['outage_start_dt'] = pd.to_datetime(
    df['OUTAGE.START.DATE'].astype(str) + ' ' + df['OUTAGE.START.TIME'].astype(str),
    format=dt_format,
    errors='coerce'
)
# 3. Combine `OUTAGE.RESTORATION.DATE` and `OUTAGE.RESTORATION.TIME` into a single
    column `outage_restoration_dt`
df['outage_restoration_dt'] = pd.to_datetime(
    df['OUTAGE.RESTORATION.DATE'].astype(str) + ' ' +
    df['OUTAGE.RESTORATION.TIME'].astype(str),
    format=dt_format,
    errors='coerce'
)

# 4. Drop the original date and time columns
time_cols_to_drop = [
    'OUTAGE.START.DATE', 'OUTAGE.START.TIME',
    'OUTAGE.RESTORATION.DATE', 'OUTAGE.RESTORATION.TIME'
]
df = df.drop(columns=time_cols_to_drop)

# 5. Drop rows where datetime values are null
df = df.dropna(subset=['outage_start_dt', 'outage_restoration_dt'])

# 6. Fill CLIMATE.REGION null values with the value from U.S._STATE + " Region"
df['CLIMATE.REGION'] = df['CLIMATE.REGION'].fillna(df['U.S._STATE'] + " Region")

# 7. Fill NaN values in with the parent CAUSE.CATEGORY.DETAIL + unknown
df['CAUSE.CATEGORY.DETAIL'] =
    df['CAUSE.CATEGORY.DETAIL'].fillna(df['CAUSE.CATEGORY'] + " unknown")

# 8. Create a binary classifier column based off of HURRICANE.NAMES NaNs
df['is_hurricane'] = df['HURRICANE.NAMES'].notnull()

# 9. drop NaNs in `electricity_economics` columns
df = df.dropna(subset=['TOTAL.PRICE'])

# 10. Data Cleaning: Fill NaN with 0 for `POPDEN_UC` and `POPDEN_RURAL` since
    District of Columbia is all city
# Find the indices of the rows with nulls
mask = df[column_categories['demographics_and_land_use']].isnull().any(axis=1)
cols = column_categories['demographics_and_land_use']

# Update those specific spots directly
df.loc[mask, cols] = df.loc[mask, cols].fillna(0)
```

1.2.4.1 Check Lingering NaN Values

- 1. Check for any remaining NaN in the dataset
- 2. Get count of NaN for each column if they exist

- 3. Check `.info()` for NaN

```
# 1. Check for any remaining NaN in the dataset
null_summary = df.isnull().sum()
total_nulls = null_summary.sum()

print(f"Total remaining NaN: {total_nulls}")

# 2. Get count of NaN for each column if they exist
if total_nulls > 0:
    print("\nColumns with remaining NaN:")
    print(null_summary[null_summary > 0])

# 3. Check `.info()` for NaN
print("\nDataset Info (with NaN details):")
df.info()
```

```
Total remaining NaN: 2459

Columns with remaining NaN:
HURRICANE.NAMES       1385
DEMAND.LOSS.MW         663
CUSTOMERS.AFFECTED     411
dtype: int64

Dataset Info (with NaN details):
<class 'pandas.core.frame.DataFrame'>
Index: 1456 entries, 0 to 1532
Data columns (total 54 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   YEAR                   1456 non-null   int64
 1   MONTH                  1456 non-null   float64
 2   U.S._STATE             1456 non-null   object
 3   POSTAL.CODE            1456 non-null   object
 4   NERC.REGION            1456 non-null   object
 5   CLIMATE.REGION         1456 non-null   object
 6   ANOMALY.LEVEL          1456 non-null   float64
 7   CLIMATE.CATEGORY       1456 non-null   object
 8   CAUSE.CATEGORY         1456 non-null   object
 9   CAUSE.CATEGORY.DETAIL  1456 non-null   object
 10  HURRICANE.NAMES        71 non-null     object
 11  OUTAGE.DURATION        1456 non-null   float64
 12  DEMAND.LOSS.MW         793 non-null    float64
 13  CUSTOMERS.AFFECTED     1045 non-null   float64
 14  RES.PRICE              1456 non-null   float64
 15  COM.PRICE              1456 non-null   float64
 16  IND.PRICE              1456 non-null   float64
 17  TOTAL.PRICE            1456 non-null   float64
 18  RES.SALES              1456 non-null   float64
 19  COM.SALES              1456 non-null   float64
 20  IND.SALES              1456 non-null   float64
 21  TOTAL.SALES            1456 non-null   float64
 22  RES.PERCEN             1456 non-null   float64
```

```
 23  COM.PERCEN             1456 non-null   float64
 24  IND.PERCEN             1456 non-null   float64
 25  RES.CUSTOMERS          1456 non-null   int64
 26  COM.CUSTOMERS          1456 non-null   int64
 27  IND.CUSTOMERS          1456 non-null   int64
 28  TOTAL.CUSTOMERS        1456 non-null   int64
 29  RES.CUST.PCT           1456 non-null   float64
 30  COM.CUST.PCT           1456 non-null   float64
 31  IND.CUST.PCT           1456 non-null   float64
 32  PC.REALGSP.STATE       1456 non-null   int64
 33  PC.REALGSP.USA         1456 non-null   int64
 34  PC.REALGSP.REL         1456 non-null   float64
 35  PC.REALGSP.CHANGE      1456 non-null   float64
 36  UTIL.REALGSP           1456 non-null   int64
 37  TOTAL.REALGSP          1456 non-null   int64
 38  UTIL.CONTRI            1456 non-null   float64
 39  PI.UTIL.OFUSA          1456 non-null   float64
 40  POPULATION             1456 non-null   int64
 41  POPPCT_URBAN           1456 non-null   float64
 42  POPPCT_UC              1456 non-null   float64
 43  POPDEN_URBAN           1456 non-null   float64
 44  POPDEN_UC              1456 non-null   float64
 45  POPDEN_RURAL           1456 non-null   float64
 46  AREAPCT_URBAN          1456 non-null   float64
 47  AREAPCT_UC             1456 non-null   float64
 48  PCT_LAND               1456 non-null   float64
 49  PCT_WATER_TOT          1456 non-null   float64
 50  PCT_WATER_INLAND       1456 non-null   float64
 51  outage_start_dt        1456 non-null   datetime64[ns]
 52  outage_restoration_dt  1456 non-null   datetime64[ns]
 53  is_hurricane           1456 non-null   bool
dtypes: bool(1), datetime64[ns](2), float64(33), int64(10), object(8)
memory usage: 615.7+ KB
```

1.2.4.2 Check Lingering NaN Values: Comments

- **Data Retention:** Removed approximately 5% of the original records during the cleaning process.
- **Impact Metrics:** After cleaning, the dataset retains
  - 793 valid observations for `DEMAND.LOSS.MW`
  - 1,045 for `CUSTOMERS.AFFECTED`

1.2.5 Univariate Analysis

1.2.5.1 Univariate Plot 1 and Plot 2

- Below we look at the frequency of outages across different cause categories and the overall distribution of outage lengths to identify common patterns and potential outliers

```
# Plot 1: Distribution of Outage Duration
fig_uni_1 = px.histogram(df, x='OUTAGE.DURATION', nbins=50,
                         title='Distribution of Outage Durations (Minutes)',
                         labels={'OUTAGE.DURATION': 'Outage Duration (Mins)'})
fig_uni_1.write_html('../data/docs/img/plots/step2.1_outage_duration_dist.html',
    include_plotlyjs='cdn')
fig_uni_1.show()

# Plot 2: Distribution of Outage Causes
fig_uni_2 = px.bar(df['CAUSE.CATEGORY'].value_counts().reset_index(),
                   x='CAUSE.CATEGORY', y='count',
                   title='Count of Outages by Cause Category')
fig_uni_2.write_html('../data/docs/img/plots/step2.2_cause_category_counts.html',
    include_plotlyjs='cdn')
fig_uni_2.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.2.5.2 Univariate Plot 1 and Plot 2: Comments

- **Duration Distribution:** `OUTAGE.DURATION` is heavily right-skewed, with most outages lasting relatively short periods and a small number of extreme events extending beyond 100,000 minutes.
- **Cause Frequency:** `CAUSE.CATEGORY` is dominated by Severe Weather and Intentional Attack events, while the remaining categories appear much less frequently.

1.2.5.3 Univariate Plot 3

- Below, we visualize the geographic footprints of the various NERC regions across the U.S. to see how power outages are distributed through the lens of the electrical grid's organizational boundaries rather than just state lines.

```
# 1. Prepare NERC Data and calculate totals
# Get unique NERC regions
nerc_list = sorted(df['NERC.REGION'].dropna().unique())
num_regions = len(nerc_list)

# Pre-calculate total outages per NERC region for subplot titles/hover
nerc_totals = df.groupby('NERC.REGION').size().reset_index(name='Region_Total')

# 2. Dynamic Grid Dimensions
cols = 3
rows = math.ceil(num_regions / cols)

# 3. High-Contrast Palette
nerc_palette = px.colors.qualitative.Alphabet
```

```
color_map_nerc = {region: nerc_palette[i % len(nerc_palette)] for i, region in
    enumerate(nerc_list)}

# 4. Initialize Subplot Grid
# We include the region's total outage count in the subplot title
fig = make_subplots(
    rows=rows, cols=cols,
    subplot_titles=[f"{r} (Total:
    {nerc_totals[nerc_totals['NERC.REGION']==r]['Region_Total'].values[0]})" for r
    in nerc_list],
    specs=[[{"type": "choropleth"}] * cols for _ in range(rows)],
    vertical_spacing=0.07,
    horizontal_spacing=0.02
)

# 5. Add Isolated NERC Maps with Dual-Count Hover
for i, region in enumerate(nerc_list):
    curr_row = (i // cols) + 1
    curr_col = (i % cols) + 1

    # Filter data for this specific NERC region
    region_df = df[df['NERC.REGION'] == region]
    reg_total = nerc_totals[nerc_totals['NERC.REGION'] ==
    region]['Region_Total'].values[0]

    # Calculate outages per state WITHIN this specific region
    state_counts = region_df.groupby(['POSTAL.CODE',
    'U.S._STATE']).size().reset_index(name='State_In_Region_Count')

    if not state_counts.empty:
        fig.add_trace(
            go.Choropleth(
                locations=state_counts['POSTAL.CODE'],
                z=[1] * len(state_counts),
                locationmode='USA-states',
                colorscale=[[0, color_map_nerc[region]], [1,
    color_map_nerc[region]]],
                showscale=False,
                name=region,
                # customdata: [State Name, Count in this Region, Total Region Count]
                customdata=list(zip(state_counts['U.S._STATE'],
                                    state_counts['State_In_Region_Count'],
                                    [reg_total]*len(state_counts))),
                hovertemplate=(
                    "<b>%{customdata[0]}</b><br>" +
                    f"NERC Grid: {region}<br>" +
                    "Outages in this Grid: %{customdata[1]}<br>" +
                    "Total Grid Outages: %{customdata[2]}<extra></extra>"
                )
            ),
            row=curr_row, col=curr_col
        )

# 6. Final Layout Adjustments
```

```
fig.update_layout(
    height=450 * rows,
    width=1100,
    title_text="U.S. Outage Analysis: NERC Grid Footprints & Localized Statistics",
    title_x=0.5,
    template="plotly_white",
    margin=dict(l=10, r=10, t=100, b=10)
)

# Hide background land to show only states with data
fig.update_geos(
    scope="usa",
    projection_type="albers usa",
    showland=False,
    showcoastlines=False,
    showframe=False,
    fitbounds="locations"
)
fig.write_html('../data/docs/img/plots/univariate_plt3.html', include_plotlyjs='cdn')
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.2.5.4 Univariate Plot 3: Comments

- **Regional Footprints:** The NERC maps show that outage events are organized across multi-state grid regions rather than being confined to individual state boundaries.
- **Frequency Variation:** Outage counts vary substantially across NERC regions, with some regions appearing far more often in the dataset than others.

1.2.5.5 Univariate Plot 4

- Below, we shift our focus to Climate Regions. This dashboard combines horizontal bar charts for each region with a comprehensive map to show which states are the primary contributors to outage totals within specific environmental zones.

```
# Prepare map_data with required columns
map_data = df[['CLIMATE.REGION', 'POSTAL.CODE', 'U.S._STATE']].dropna()

# State outages within each region
state_counts = (
    map_data.groupby(['CLIMATE.REGION', 'POSTAL.CODE', 'U.S._STATE'])
    .size()
    .reset_index(name='State_Outages')
)

# Total outages per region
region_totals = (
    map_data.groupby('CLIMATE.REGION')
    .size()
```

```
    .reset_index(name='Region_Total')
)

# Merge both into map_data
map_data = state_counts.merge(region_totals, on='CLIMATE.REGION')

# 1. Setup Colors and Regions
regions = sorted(df['CLIMATE.REGION'].dropna().unique())
custom_colors = ["#E6194B", "#3CB44B", "#FFE119", "#4363D8", "#F58231",
                 "#911EB4", "#46F0F0", "#F032E6", "#BCF60C", "#008080"]
color_map = {region: custom_colors[i % len(custom_colors)] for i, region in
    enumerate(regions)}

# 2. Initialize 6-Row Grid
fig = make_subplots(
    rows=6, cols=3,
    row_heights=[0.12, 0.12, 0.12, 0.21, 0.21, 0.21],
    vertical_spacing=0.06,
    # Ensure titles only apply to the first 9 bar charts + the 1 map
    subplot_titles=regions[:9] + ["Large-Scale Regional Mapping"],
    specs=[[{"type": "bar"}]*3, [{"type": "bar"}]*3, [{"type": "bar"}]*3,
           [{"type": "choropleth", "colspan": 3, "rowspan": 3}, None, None],
           [None, None, None], [None, None, None]]
)

# 3. Add Bar Charts (Limited to first 9 to avoid grid conflict)
for i, region in enumerate(regions[:9]):
    row, col = (i // 3) + 1, (i % 3) + 1
    region_df = df[df['CLIMATE.REGION'] == region]
    counts =
    region_df['U.S._STATE'].value_counts().reset_index().sort_values('count')

    fig.add_trace(go.Bar(
        y=counts['U.S._STATE'], x=counts['count'], orientation='h',
        marker=dict(color=color_map[region]), name=region, showlegend=False,
        hovertemplate="<b>%{y}</b><br>State Outages: %{x}<extra></extra>"
    ), row=row, col=col)

# 4. Add Large-Scale Choropleth (Always row 4, col 1)
for region in regions:
    region_states = map_data[map_data['CLIMATE.REGION'] == region]
    if region_states.empty: continue

    fig.add_trace(go.Choropleth(
        locations=region_states['POSTAL.CODE'],
        z=[1] * len(region_states),
        locationmode='USA-states',
        colorscale=[[0, color_map[region]], [1, color_map[region]]],
        showscale=False,
        name=region,
        showlegend=True,
        marker_line_color='white',
        marker_line_width=0.5,
        customdata=region_states[['U.S._STATE', 'State_Outages',
```

```
    'Region_Total']].values,
        hovertemplate=(
            "<b>%{customdata[0]}</b><br>" +
            "Region: " + region + "<br>" +
            "State Outages: %{customdata[1]}<br>" +
            "Total Region Outages: %{customdata[2]}<extra></extra>"
        )
    ), row=4, col=1)

# 5. Final Layout
fig.update_layout(
    height=1600, width=1100, title_text="U.S. Outage Analysis: Regional Statistics",
    title_x=0.5, template="plotly_white",
    legend=dict(orientation="h", yanchor="bottom", y=0.46, xanchor="center", x=0.5)
)

fig.update_geos(scope="usa", projection_type="albers usa")
fig.write_html('../data/docs/img/plots/univariate_plt4.html', include_plotlyjs='cdn')
fig.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.2.5.6 Univariate Plot 4: Comments

- **Climate Hotspots:** The Northeast and South climate regions contain the largest numbers of outage events in the dataset.
- **Intra-Regional Concentration:** Within several climate regions, a small number of states account for a large share of recorded outages, such as California in the West and Texas in the South.

1.2.6 Bivariate Analysis

1.2.6.1 Bivariate Plot 1

- Below, we examine the relationship between climate zones and the length of power disruptions. By using a log-scale box plot, we can compare the "typical" outage length against the extreme outliers across different geographic environments.

```
# Outage Duration by Climate Region Box Plot (Adjusted for 0s)
df_box = df.dropna(subset=['OUTAGE.DURATION', 'CLIMATE.REGION']).copy()

# Add 1 to duration to prevent log(0) errors and allow 0s to appear at the bottom
df_box['OUTAGE.DURATION.ADJUSTED'] = df_box['OUTAGE.DURATION'] + 1

fig_bi_2 = px.box(df_box, x='CLIMATE.REGION', y='OUTAGE.DURATION.ADJUSTED',
                  title='Outage Duration Across Climate Regions',
                  color='CLIMATE.REGION')

# Update y-axis to log scale and adjust the label to reflect the mathematical change
```

```
fig_bi_2.update_layout(
    yaxis_type="log",
    yaxis_title="OUTAGE.DURATION (mins + 1)"
)
fig_bi_2.write_html('../data/docs/img/plots/bivariate_plt5.html',
    include_plotlyjs='cdn')
fig_bi_2.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.2.6.2 Bivariate Plot 1: Comments

- **Regional Consistency:** Median outage duration is fairly similar across many climate regions, suggesting that typical restoration times do not differ dramatically by climate zone.
- **Extreme Outliers:** Regions such as the Northeast and South show more extreme long-duration outliers, indicating a heavier upper tail in outage duration.
- **Log-Scale Necessity:** Using `log(mins + 1)` makes it possible to compare very short and very long outages on the same figure without the largest events dominating the view.

1.2.6.3 Bivariate Plot 2

- Below, we investigate whether the time of year has an impact on how long it takes to restore power. By grouping the durations by month, we can see if seasonal weather patterns-like winter storms or summer heatwaves-lead to longer recovery times.

```
# 1. Clean and prepare data
df_month = df.dropna(subset=['OUTAGE.DURATION', 'outage_start_dt']).copy()

# 2. Extract Month and apply +1 adjustment (same logic as your working plot)
df_month['START.MONTH'] = df_month['outage_start_dt'].dt.month_name()
df_month['OUTAGE.DURATION.ADJUSTED'] = df_month['OUTAGE.DURATION'] + 1

# 3. Build the plot - MATCH the working Climate Region pattern exactly
fig_bi_1 = px.box(
    df_month,
    x='START.MONTH',
    y='OUTAGE.DURATION.ADJUSTED',
    title='Distribution of Outage Duration by Month (Log Scale)',
    color='START.MONTH',
    category_orders={"START.MONTH": [
        "January", "February", "March", "April", "May", "June",
        "July", "August", "September", "October", "November", "December"
    ]}
)

# 4. Log scale + title - same 2-line pattern as your working plot
fig_bi_1.update_layout(
    yaxis_type="log",
    yaxis_title="OUTAGE.DURATION (mins + 1)",
```

```
    showlegend=False
)

# 5. Display and Export
fig_bi_1.show()
fig_bi_1.write_html('../data/docs/img/plots/bivariate_plt6.html',
    include_plotlyjs='cdn')
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.2.6.4 Bivariate Plot 2: Comments

- **Seasonal Variation:** Outage duration varies by month primarily in the spread and frequency of extreme outliers rather than in large shifts in the median.
- **Monthly Stability:** Median outage duration remains in a relatively narrow range across months, suggesting that typical outages are broadly similar throughout the year.
- **Log-Scale Visuals:** The log scale is necessary because outage durations span from near-zero events to outages lasting many days or weeks.

1.2.7 Interesting Aggregates

- Below, we aggregate the data into a pivot table comparing outage duration across cause categories and climate regions. This helps identify whether certain types of disruptions are associated with longer recovery times in particular environmental contexts.

```
# Group by Climate Region and Cause, looking at median duration and average
    customers affected
agg_table = df.groupby(['CLIMATE.REGION', 'CAUSE.CATEGORY']).agg({
    'OUTAGE.DURATION': 'median',
    'CUSTOMERS.AFFECTED': 'mean'
}).reset_index().round(2)

# Pivot the table for better readability
pivot_table = agg_table.pivot(index='CLIMATE.REGION', columns='CAUSE.CATEGORY',
    values='OUTAGE.DURATION')
display(pivot_table)
```

CAUSE.CATEGORY CLIMATE.REGION	equipment failure	fuel supply emergency	intentional attack	islanding	public appea
Central	149.0	7500.5	50.0	96.0	1410.0
East North Central	761.0	13564.0	648.5	1.0	733.0
Hawaii Region	NaN	NaN	NaN	NaN	NaN
Northeast	159.0	12240.0	1.0	881.0	2760.0
Northwest	702.0	1.0	74.0	21.0	898.0
South	227.0	20160.0	100.0	493.5	422.0
Southeast	308.5	NaN	95.5	NaN	1337.0

CAUSE.CATEGORY CLIMATE.REGION	equipment failure	fuel supply emergency	intentional attack	islanding	public appea
Southwest	35.0	76.0	56.0	2.0	2275.0
West	269.0	882.5	108.0	128.5	420.0
West North Central	61.0	NaN	0.5	56.0	439.5

```
#### Visualize the pivot table as a heatmap
import plotly.express as px
fig_heat = px.imshow(pivot_table,
                     labels=dict(x="Cause Category", y="Climate Region",
    color="Median Duration"),
                     title="Heatmap: Median Outage Duration (Mins)")
fig_heat.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.3 Step 3: Assessment of Missingness

1.3.1 NMAR Analysis

We examine two columns with significant missingness to classify their missingness mechanisms.

1.3.1.1 Example 1: `HURRICANE.NAMES` - Missing by Design (MD)

`HURRICANE.NAMES` has approximately 95% missing values. This is **Missing by Design**: a hurricane name only exists when the outage was caused by a hurricane. The missingness is entirely deterministic from `CAUSE.CATEGORY.DETAIL`.

1.3.1.2 Example 2: `DEMAND.LOSS.MW` - NMAR (Not Missing At Random)

`DEMAND.LOSS.MW` has approximately 46% missing values. Unlike `HURRICANE.NAMES`, this column should have a value for every outage. The missingness cannot be explained by design.

NMAR Reasoning: `DEMAND.LOSS.MW` measures peak megawatt demand lost during an outage. Reporting this value requires SCADA (Supervisory Control and Data Acquisition) metering infrastructure. Smaller and rural utilities often lack this metering capability. The missingness is therefore related to the **unobserved value itself**: small losses come from small utilities that can't measure them. This is **NMAR**.

```python
# Verify: HURRICANE.NAMES is Missing by Design
print('CAUSE.CATEGORY values when HURRICANE.NAMES is non-null:')
display(
    df.loc[pd.notna(df['HURRICANE.NAMES']), ['CAUSE.CATEGORY',
    'CAUSE.CATEGORY.DETAIL']]
    .drop_duplicates()
)

# Edge case: hurricane-caused outages with missing name?
mask = df['HURRICANE.NAMES'].isna() & (df['CAUSE.CATEGORY.DETAIL'] == 'hurricanes')
n_edge = mask.sum()
print(f'\nHurricane-caused outages with missing name: {n_edge}
    ({n_edge/len(df):.1%}) , safely negligible.')
print('Conclusion: HURRICANE.NAMES is Missing by Design (MD), not NMAR.')
```

```
CAUSE.CATEGORY values when HURRICANE.NAMES is non-null:

Hurricane-caused outages with missing name: 2 (0.1%) , safely negligible.
Conclusion: HURRICANE.NAMES is Missing by Design (MD), not NMAR.
```

	CAUSE.CATEGORY	CAUSE.CATEGORY.DETAIL
71	severe weather	hurricanes

1.3.2 Missingness Dependency Tests

- Now, we test if the missingness of `DEMAND.LOSS.MW` **depends on** or **does not depend on** other observed columns.
- **Total Variation Distance (TVD):** is the test statistic for categorical comparisons, with 1,000 permutations.

```python
# define a function to calculate Total Variation Distance (TVD) between two
    categorical distributions
def tvd(dist1, dist2):
    """Total Variation Distance between two categorical distributions."""
    all_cats = set(dist1.index) | set(dist2.index)
    dist1 = dist1.reindex(all_cats, fill_value=0)
    dist2 = dist2.reindex(all_cats, fill_value=0)
    return 0.5 * np.abs(dist1 - dist2).sum()

def missingness_permutation_test(df, missing_col, test_col, n_perms=1000):
    """Permutation test: is distribution of test_col different when missing_col is
    null vs non-null?"""
    is_missing = df[missing_col].isna()
    dist_missing = df.loc[is_missing, test_col].value_counts(normalize=True)
    dist_present = df.loc[~is_missing, test_col].value_counts(normalize=True)
    observed_tvd = tvd(dist_missing, dist_present)

    perm_tvds = np.zeros(n_perms)
    for i in range(n_perms):
```

```
        shuffled = np.random.permutation(is_missing.values)
        d1 = df.loc[shuffled, test_col].value_counts(normalize=True)
        d2 = df.loc[~shuffled, test_col].value_counts(normalize=True)
        perm_tvds[i] = tvd(d1, d2)

    p_value = np.mean(perm_tvds >= observed_tvd)
    return observed_tvd, p_value, perm_tvds


# Define Test 1
# DEMAND.LOSS.MW missingness vs CAUSE.CATEGORY
np.random.seed(42)
obs1, p1, perms1 = missingness_permutation_test(df, 'DEMAND.LOSS.MW',
    'CAUSE.CATEGORY', n_perms=1000)

print('Test 1: DEMAND.LOSS.MW and CAUSE.CATEGORY')
print(f'Observed TVD: {obs1:.4f}, p-value: {p1:.4f}')
if p1 < 0.05:
    print('REJECT H0: The distribution of CAUSE.CATEGORY differs significantly
    between rows where DEMAND.LOSS.MW is missing vs present.')
else:
    print('FAIL TO REJECT H0: No significant difference in CAUSE.CATEGORY
    distribution based on DEMAND.LOSS.MW missingness.')
# Distribution comparison plot
miss_mask = df['DEMAND.LOSS.MW'].isna()
compare_df = df[['CAUSE.CATEGORY']].copy()
compare_df['DEMAND.LOSS.MW Status'] = np.where(miss_mask, 'Missing', 'Present')

fig_m1 = px.histogram(compare_df, x='CAUSE.CATEGORY', color='DEMAND.LOSS.MW Status',
                      barmode='group', histnorm='percent',
                      title='Distribution of CAUSE.CATEGORY: DEMAND.LOSS.MW Missing
    vs Present',
                      color_discrete_map={'Missing': '#EF553B', 'Present':
    '#636EFA'})
fig_m1.update_layout(xaxis_title='Cause Category', yaxis_title='Percentage (%)',
    xaxis_tickangle=-30)
fig_m1.write_html('../data/docs/img/plots/step3.1_missingness_cause_category.html',
    include_plotlyjs='cdn')
fig_m1.show()

# Permutation null distribution
fig_p1 = go.Figure()
fig_p1.add_trace(go.Histogram(x=perms1, nbinsx=40, name='Permuted TVDs',
    marker_color='#636EFA', opacity=0.7))
fig_p1.add_vline(x=obs1, line_dash='dash', line_color='red',
    annotation_text=f'Observed TVD = {obs1:.4f}')
fig_p1.update_layout(title=f'Permutation Test: DEMAND.LOSS.MW ~ CAUSE.CATEGORY (p =
    {p1:.4f})',
                     xaxis_title='Total Variation Distance', yaxis_title='Count')
fig_p1.write_html('../data/docs/img/plots/step3.2_tvd_permutation_cause.html',
    include_plotlyjs='cdn')
fig_p1.show()
```

```
Test 1: DEMAND.LOSS.MW and CAUSE.CATEGORY
```

```
Observed TVD: 0.1844, p-value: 0.0000
REJECT H0: The distribution of CAUSE.CATEGORY differs significantly between rows
    where DEMAND.LOSS.MW is missing vs present.
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
# Define Test 2: DEMAND.LOSS.MW missingness vs CLIMATE.CATEGORY
test2_df = df.dropna(subset=['CLIMATE.CATEGORY']).copy()

np.random.seed(42)
obs2, p2, perms2 = missingness_permutation_test(test2_df, 'DEMAND.LOSS.MW',
    'CLIMATE.CATEGORY', n_perms=1000)

print('Test 2: DEMAND.LOSS.MW and CLIMATE.CATEGORY')
print(f'Observed TVD: {obs2:.4f}, P-value: {p2:.4f}')
if p2 < 0.05:
    print('REJECT H0: The distribution of CLIMATE.CATEGORY differs significantly
    between rows where DEMAND.LOSS.MW is missing vs present.')
else:
    print('FAIL TO REJECT H0: No significant difference in CLIMATE.CATEGORY
    distribution based on DEMAND.LOSS.MW missingness.')
# Distribution comparison
miss_mask2 = test2_df['DEMAND.LOSS.MW'].isna()
compare_df2 = test2_df[['CLIMATE.CATEGORY']].copy()
compare_df2['DEMAND.LOSS.MW Status'] = np.where(miss_mask2, 'Missing', 'Present')

fig_m2 = px.histogram(compare_df2, x='CLIMATE.CATEGORY', color='DEMAND.LOSS.MW
    Status',
                      barmode='group', histnorm='percent',
                      title='Distribution of CLIMATE.CATEGORY: DEMAND.LOSS.MW
    Missing vs Present',
                      color_discrete_map={'Missing': '#EF553B', 'Present':
    '#636EFA'})
fig_m2.update_layout(xaxis_title='Climate Category', yaxis_title='Percentage (%)')
fig_m2.write_html('../data/docs/img/plots/step3.3_missingness_climate_category.html',
    include_plotlyjs='cdn')
fig_m2.show()

# Permutation null distribution
fig_p2 = go.Figure()
fig_p2.add_trace(go.Histogram(x=perms2, nbinsx=40, name='Permuted TVDs',
    marker_color='#636EFA', opacity=0.7))
fig_p2.add_vline(x=obs2, line_dash='dash', line_color='red',
    annotation_text=f'Observed TVD = {obs2:.4f}')
fig_p2.update_layout(title=f'Permutation Test: DEMAND.LOSS.MW ~ CLIMATE.CATEGORY (p
    = {p2:.4f})',
                     xaxis_title='Total Variation Distance', yaxis_title='Count')
fig_p2.write_html('../data/docs/img/plots/step3.4_tvd_permutation_climate.html',
    include_plotlyjs='cdn')
fig_p2.show()
```

```
Test 2: DEMAND.LOSS.MW and CLIMATE.CATEGORY
Observed TVD: 0.0371, P-value: 0.2850
FAIL TO REJECT H0: No significant difference in CLIMATE.CATEGORY distribution based
    on DEMAND.LOSS.MW missingness.
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.3.3 Missingness Dependency: Interpretation

- **Test 1: `CAUSE.CATEGORY` (TVD ≈ 0.18, p ≈ 0.000):**
  - **Reject** H_0 at $\alpha = 0.05$
  - The missingness of `DEMAND.LOSS.MW` depends on `CAUSE.CATEGORY`.
  - Intentional attacks are overrepresented among missing rows (~34%) vs. present rows (~22%), while system operability events are overrepresented among present rows.
- **Test 2: `CLIMATE.CATEGORY` (TVD ≈ 0.04, p ≈ 0.285):**
  - **Fail to reject** H_0 at α =0.05
  - The missingness of `DEMAND.LOSS.MW` does not depend on `CLIMATE.CATEGORY`.
  - Climate conditions influence outage occurrence and severity, but not whether the utility can report megawatt losses.

1.3.4 Summary

Column	Missingness Type	Evidence
`HURRICANE.NAMES`	**Missing by Design (MD)**	Nearly all missing values occur when the outage was not hurricane-related.
`DEMAND.LOSS.MW`	**NMAR**	Missingness is substantively linked to whether utilities can measure/report MW loss, which may relate to the unobserved value itself.
`DEMAND.LOSS.MW` ~ `CAUSE.CATEGORY`	**MAR** (conditional)	p ~ 0.000 ; missingness depends on outage cause
`DEMAND.LOSS.MW` ~ `CLIMATE.CATEGORY`	**MCAR** (conditional)	p ~ 0.285 ; missingness is independent of climate

- **Future modeling:** Any future imputation of `DEMAND.LOSS.MW` should condition on `CAUSE.CATEGORY` (since missingness depends on it) but does not need to account for `CLIMATE.CATEGORY`.

1.4 Step 4: Hypothesis Testing

1.4.1 Hypothesis Test: Does Outage Duration Vary by State Within the Same NERC Region?

- If states within the same NERC reliability region experience statistically different outage durations, it implies that state level factors matter beyond the broad regional grid classification.
- The test is within the **WECC** area
  - Western Electricity Coordinating Council (WECC): The largest and most diverse region

1.4.1.1 Null Hypothesis (H_{0})

- The mean log transformed `OUTAGE.DURATION` is the same across all states within the WECC region.

1.4.1.2 Alternative Hypothesis ($H_{1})

- At least one state's mean log-transformed `OUTAGE.DURATION` differs significantly from the others within WECC.

1.4.1.3 Test Statistic

- **F-statistic:** Via a one way ANOVA via `statsmodels.formula.api.ols`.

1.4.1.4 Significance Level: $\alpha = 0.05$

1.4.1.5 Log Transform

- Log transform is used since ANOVA assumes approximately normal residuals.
- Raw `OUTAGE.DURATION` has a skewness > 10 with extreme outliers exceeding 100,000 minutes.
- Applying `np.log1p` compresses this long tail and brings the residual distribution closer to normality.

```
# One-Way ANOVA: Outage Duration by State within WECC

# 1. Filter to WECC and drop missing durations
wecc = df[df['NERC.REGION'] == 'WECC'].dropna(subset=['OUTAGE.DURATION']).copy()

# 2. Keep states with >= 5 observations for stable estimates
state_counts = wecc['U.S._STATE'].value_counts()
valid_states = state_counts[state_counts >= 5].index.tolist()
wecc_anova_df = wecc[wecc['U.S._STATE'].isin(valid_states)].copy()

# 3. Log-transform duration
wecc_anova_df['LOG_DURATION'] = np.log1p(wecc_anova_df['OUTAGE.DURATION'])
```

```
# 4. Fit OLS model
model = ols('LOG_DURATION ~ Q("U.S._STATE")', data=wecc_anova_df).fit()
anova_table = sm.stats.anova_lm(model, typ=2)

print('ANOVA Table (WECC Region, log-transformed duration)')
display(anova_table)

p_val = anova_table.loc['Q("U.S._STATE")', 'PR(>F)']
f_val = anova_table.loc['Q("U.S._STATE")', 'F']
print(f'\nF = {f_val:.4f}, p = {p_val:.6f}')
if p_val < 0.05:
    print('REJECT H0: Significant variation in outage duration exists between states
    in WECC.')
else:
    print('FAIL TO REJECT H0: No significant difference detected.')
```

```
ANOVA Table (WECC Region, log-transformed duration)

F = 5.0488, p = 0.000001
REJECT H0: Significant variation in outage duration exists between states in WECC.
```

	sum_sq	df	F	PR(>F)
Q("U.S._STATE")	309.047877	10.0	5.04877	6.092311e-07
Residual	2479.106683	405.0	NaN	NaN

```
# Box plot: Duration by State within WECC
wecc_anova_df['DURATION.ADJUSTED'] = wecc_anova_df['OUTAGE.DURATION'] + 1

fig_hyp = px.box(wecc_anova_df, x='U.S._STATE', y='DURATION.ADJUSTED',
                 color='U.S._STATE',
                 title='Outage Duration by State within WECC Region',
                 labels={'U.S._STATE': 'State', 'DURATION.ADJUSTED': 'Outage
    Duration (mins + 1)'},
                 category_orders={'U.S._STATE': sorted(valid_states)})
fig_hyp.update_layout(yaxis_type='log', yaxis_title='Outage Duration (mins + 1, log
    scale)',
                      showlegend=False, xaxis_tickangle=-30)
fig_hyp.write_html('../data/docs/img/plots/step4.1_duration_by_state_wecc.html',
    include_plotlyjs='cdn')
fig_hyp.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.4.2 Step 4 Conclusions

- **The Result: REJECT H_0**
  - $p \approx 0.000$

- Even though these states share the same grid (WECC), their outage times are not similar.
- **Example:** California has prolonged outages (*potential causes: wildfires/safety shutoffs*), while Utah and Colorado outages are more brief.
  - **Note:** Knowing the region isn't enough and local state factors matter more.
  - **Note:** Since state lines significantly influence outage length, we've made `U.S._STATE` a core ingredient in our model to ensure the predictions are context-aware.

1.5 Step 5: Framing a Prediction Problem

The goal of this prediction task is to estimate outage severity using information available at or shortly after the start of an outage. Although Step 4 tested differences in `OUTAGE.DURATION`, that analysis showed that outage behavior varies meaningfully by `U.S._STATE` even within the same NERC region, which support including location-based features in the predictive model. In this step, we define a regression problem whose response variable is `CUSTOMERS.AFFECTED`, our main operational measure of outage severity.

Problem Type: Regression

Response Variable: `CUSTOMERS.AFFECTED`
- `CUSTOMERS.AFFECTED` represents the number of customers who lost power during the outage. - We selected `CUSTOMERS.AFFECTED` because it is the most direct measure of outage severity from a public-safety perspective. - It is also more actionable than many other outage descriptors, since large customer-impact events are the ones utilities and emergency planners are most concerned with identifying early.

1.5.0.1 Features Available at Time of Prediction

We restrict attention to features that are known at the time of prediction, or that would reasonably be available shortly after outage onset. This avoids leakage from information that would only be known after the event has fully unfolded.

Feature	Type	Available	Rationale
`U.S._STATE`	Nominal	True	Location is known immediately and Step 4 suggests state-level differences matter beyond broad grid region
`NERC.REGION`	Nominal	True	Grid region is fixed per location
`CLIMATE.CATEGORY`	Nominal	True	Climate episode is known for the period
`CAUSE.CATEGORY`	Nominal	Usually	Cause is often identified early in outage reporting, though not always perfectly at onset

Feature	Type	Available	Rationale
ANOMALY.LEVEL	Quantitative	True	ONI-based climate anomaly information is pre-published
POPULATION	Quantitative	True	Census-based state population is pre-known
POPPCT_URBAN	Quantitative	True	Urbanization level is pre-known from census data
is_hurricane	Binary	True	Hurricane conditions can be tracked in real time

1.5.0.2 Evaluation Metric

RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

We use RMSE to evaluate model performance on the test set.

Why RMSE is appropriate: - RMSE penalizes large prediction errors more heavily than small ones. - This is important here because missing a very large outage affecting hundreds of thousands of customers is much worse than missing a small outage by the same relative amount. - RMSE is measured in the original units of the response variable, so it is easy to interpret in terms of customers affected.

```
# Prepare data for modeling

# Select non-null CUSTOMERS.AFFECTED rows for modeling
model_df = df.dropna(subset=['CUSTOMERS.AFFECTED']).copy()

# Define features and target
nominal_features = ['U.S._STATE', 'NERC.REGION', 'CLIMATE.CATEGORY',
    'CAUSE.CATEGORY']
quantitative_features = ['ANOMALY.LEVEL', 'POPULATION', 'POPPCT_URBAN']
binary_features = ['is_hurricane']
all_features = nominal_features + quantitative_features + binary_features
target = 'CUSTOMERS.AFFECTED'

# Drop rows where any feature is NaN
model_df = model_df.dropna(subset=all_features)

print(f'Total cleaned rows:     {len(df)}')
print(f'Rows with target value: {len(model_df)} ({len(model_df)/len(df):.1%})')
print(f'Target skewness:        {model_df[target].skew():.2f}')
```

```
# Train/test split (80/20)
X = model_df[all_features]
y = model_df[target]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    random_state=42)
print(f'\nTrain set: {len(X_train)} rows')
print(f'Test set:  {len(X_test)} rows')
```

```
Total cleaned rows:     1456
Rows with target value: 1045 (71.8%)
Target skewness:        6.06

Train set: 836 rows
Test set:  209 rows
```

1.6 Step 6: Baseline Model

1.6.1 Baseline Model: Ridge Regression with Log-Transformed Target

- **Algorithm:** Ridge Regression (*Linear Regression with L2 regularization*)

1.6.2 Why Use Ridge?

- **The target variable is heavily skewed**
  - Note: A small number of extreme outages (up to 3.2M customers) would pull a regular model off track. Predicting on a log scale smooths this out so the model isn't dominated by outliers.
- **There are many correlated, one-hot encoded columns** (like ~50 state dummies)
  - Note: Ridge shrinks their coefficients rather than letting any one of them blow up, which helps the model generalise instead of memorising the training data.

1.6.3 Feature Classification

Feature	Type	Encoding
`U.S._STATE`	Nominal	OneHotEncoder
`NERC.REGION`	Nominal	OneHotEncoder
`CLIMATE.CATEGORY`	Nominal	OneHotEncoder
`CAUSE.CATEGORY`	Nominal	OneHotEncoder
`ANOMALY.LEVEL`	Quantitative	StandardScaler
`POPULATION`	Quantitative	StandardScaler
`POPPCT_URBAN`	Quantitative	StandardScaler
`is_hurricane`	Binary (Nominal)	Passthrough

```
# Train and Evaluate Baseline Model (Ridge Regression)

# 1. Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('nominal', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
    nominal_features),
        ('quantitative', StandardScaler(), quantitative_features),
        ('binary', 'passthrough', binary_features)
    ]
)

# 2. Log-transform the target (handles skewness + prevents negative predictions)
y_train_log = np.log1p(y_train)
y_test_log = np.log1p(y_test)

# 3. Baseline pipeline: preprocessor → Ridge
baseline_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('model', Ridge(alpha=1.0))
])
baseline_pipeline.fit(X_train, y_train_log)

# 4. Predict on log scale, then exponentiate back
y_pred_log = baseline_pipeline.predict(X_test)
y_pred_baseline = np.expm1(y_pred_log)  # inverse of log1p
y_pred_baseline = np.maximum(y_pred_baseline, 0)  # floor at 0

# 5. Evaluate on ORIGINAL scale
rmse_baseline = np.sqrt(mean_squared_error(y_test, y_pred_baseline))
r2_baseline = r2_score(y_test, y_pred_baseline)

print('=' * 55)
print('Baseline Model: Ridge Regression (Log-Target)')
print('=' * 55)
print(f'RMSE:  {rmse_baseline:>12,.0f} customers')
print(f'R²:    {r2_baseline:>12.4f}')
print(f'Mean:  {y_test.mean():>12,.0f} (test set average)')
print(f'Std:   {y_test.std():>12,.0f} (test set std)')
print('=' * 55)

if rmse_baseline < y_test.std():
    print(f'\nRMSE ({rmse_baseline:,.0f}) < Std ({y_test.std():,.0f}) → Model IS
    better than predicting the mean.')
else:
    print(f'\nRMSE ({rmse_baseline:,.0f}) >= Std ({y_test.std():,.0f}) → Model is
    NOT better than predicting the mean.')

# Diagnostic: Predicted vs Actual
diag_df = pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred_baseline})

fig_base = px.scatter(diag_df, x='Actual', y='Predicted',
                      title='Baseline Model: Predicted vs Actual
```

```
    (CUSTOMERS.AFFECTED)',
                        labels={'Actual': 'Actual Customers Affected', 'Predicted':
    'Predicted'},
                        opacity=0.5)
max_val = max(diag_df['Actual'].max(), diag_df['Predicted'].max())
fig_base.add_shape(type='line', x0=0, y0=0, x1=max_val, y1=max_val,
                   line=dict(dash='dash', color='red', width=1))
fig_base.write_html('../data/docs/img/plots/step6.1_baseline_pred_vs_actual.html',
    include_plotlyjs='cdn')
fig_base.show()

# Residual plot
residuals_base = y_test.values - y_pred_baseline
fig_resid = px.scatter(x=y_pred_baseline, y=residuals_base,
                       title='Baseline Model: Residual Plot',
                       labels={'x': 'Predicted', 'y': 'Residual (Actual -
    Predicted)'},
                       opacity=0.5)
fig_resid.add_hline(y=0, line_dash='dash', line_color='red')
fig_resid.write_html('../data/docs/img/plots/step6.2_baseline_residuals.html',
    include_plotlyjs='cdn')
fig_resid.show()
```

```
========================================================
Baseline Model: Ridge Regression (Log-Target)
========================================================
RMSE:       223,420 customers
R²:         -0.0594
Mean:       131,051 (test set average)
Std:        217,593 (test set std)
========================================================

RMSE (223,420) >= Std (217,593) → Model is NOT better than predicting the mean.
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.6.3.1 Baseline Assessment

- Our Ridge Regression baseline model provides a clear starting benchmark for predicting `CUSTOMERS.AFFECTED`, but as a **linear** model it has important limitations.
  - It cannot easily capture **non-linear jumps** in outage impact as population and other drivers increase.
  - It struggles to model **combined effects** (for example, a hurricane in a large, urban state can be much worse than the same cause in a small, rural state).
  - It cannot fully learn **region- and state-specific patterns** (such as California wildfire shutoffs versus shorter winter storms in Utah or Colorado).

- On the held-out test set, the baseline Ridge model with a log-transformed target achieved an RMSE of 223,420 customers and an R^2 of -0.0594. Because the RMSE is slightly larger than the test-set standard deviation of `CUSTOMERS.AFFECTED`, this baseline only modestly improves over simply predicting the mean outage size and motivates a more flexible final model.
- **Final Model**
  The final model replaces the linear Ridge baseline with a tree-based gradient-boosting approach that can better handle non-linearities, interactions, and heterogeneous behavior across states and regions.

1.6.4 Model Comparison: Solving the Baseline Failures

Aspect	Random Forest	XGBoost / LightGBM / HGB	How It Addresses Baseline Failures
Non-Linear Jumps	Uses decision splits to capture sudden spikes in outage impact.	Uses gradient-boosted trees to focus specifically on hard-to-predict spikes and tails.	**Baseline Fix:** Stops trying to fit a single straight line through data that naturally "jumps," especially for very large outages.
Combined Effects	Splits simultaneously on multiple features (e.g., cause, climate, state) to form interaction rules.	Builds trees that explicitly correct errors where features interact (e.g., hurricane × coastal, heatwave × urban).	**Baseline Fix:** Learns that "Hurricane + Florida" or "Wildfire + California" can be much worse than either factor alone.
Region / State Patterns	Can isolate specific regions or states into their own branches of the tree.	Learns region- and state-specific patterns via many local decision rules and boosted corrections.	**Baseline Fix:** Creates different "rules" for California wildfires vs. Utah ice storms, instead of assuming one linear effect for all states.
Skewed Targets	Averages many trees, which can reduce the influence of single extreme outliers.	Focuses on residual errors, allowing the model to pay extra attention to very large outages without being dominated by them.	**Baseline Fix:** Reduces the risk that a 3.2M-customer outage completely distorts the fit while still trying to model the tail.
Training Speed	Can be slower for large grids or many trees.	Histogram-based gradient boosting implementations are much faster and more memory-efficient.	**Baseline Fix:** Makes it feasible to explore non-linear models and hyperparameters within the project's compute budget.

Aspect	Random Forest	XGBoost / LightGBM / HGB	How It Addresses Baseline Failures
Overfitting Risk	Naturally resistant due to bagging, but still needs tuning of depth and samples per split.	Higher overfitting risk if not regularized (depth, learning rate, leaf size, etc.).	**Baseline Fix:** With proper regularization, boosted trees achieve better flexibility than Ridge without simply memorizing the training set.
Interpretability	Provides intuitive feature importance and partial dependence plots.	Still offers feature importance, though individual trees are less interpretable.	**Baseline Fix:** Highlights which predictors (e.g., `U.S._STATE`, `CAUSE.CATEGORY`, population) drive outage severity, which the dense Ridge coefficients obscure.

1.6.5 Solving the Baseline Feature Failures

- To support a more expressive model, we also add richer features and tune capacity-controlling hyperparameters.

Type	Feature / Parameter	Why It Helps
Feature	`log_population`	Compresses the huge scale gap between small states (e.g., Wyoming 0.6M) and very large ones (e.g., California 37M), making population effects more linear on the log scale.
Feature	`state_pop_bin`	Groups states into `small`, `medium`, and `large` population buckets so the model can learn different patterns for sparsely vs. densely populated states and supports our fairness analysis.
Feature	`state_outage_rate` *(optional / exploratory)*	Captures historical average customers affected per state, flagging chronically vulnerable regions that systematically experience larger outages.

Type	Feature / Parameter	Why It Helps
Feature	`is_extreme_weather` *(optional / exploratory)*	Combines hurricanes, tornadoes, and severe winter storms into a single strong signal for extreme weather–driven events.
Feature	`outage_duration_per_customer` *(optional / exploratory)*	`OUTAGE.DURATION / CUSTOMERS.AFFECTED`; measures how concentrated impact is per customer and can highlight long, low-impact vs. short, high-impact events.
Feature	`season`	Derived from `MONTH`; allows the model to differentiate summer heatwave outages from winter ice-storm outages with different severity profiles.
Hyperparameter	`max_depth`	Controls how deep each tree can grow; shallower trees reduce variance and prevent memorizing noise, while deeper trees capture more complex interactions.
Hyperparameter	`n_estimators` / `max_leaf_nodes`	Number of trees and leaf capacity; more trees and leaves increase flexibility but require stronger regularization and careful tuning.
Hyperparameter	`learning_rate`	Governs how aggressively each boosting step corrects errors; lower learning rates produce more stable, less overfit ensembles at the cost of longer training.
Hyperparameter	`min_samples_leaf` / `min_samples_split`	Sets the minimum data required to create splits or leaves; larger values smooth predictions and guard against overfitting to tiny groups of rare outages.

1.7 Step 7: Final Model

1.7.1 Final Model Overview

For our final model, we replaced the linear Ridge baseline with a Histogram-based Gradient Boosting Regressor trained on a log-transformed `CUSTOMERS.AFFECTED` target. This gradient-boosting approach can represent non-linear relationships and interaction effects that the baseline cannot, while still producing predictions that we evaluate in the original customer units. The model uses the same core predictors as the baseline-`U.S._STATE`, `NERC.REGION`, `CLIMATE.CATEGORY`, `CAUSE.CATEGORY`, `ANOMALY.LEVEL`, `POPULATION`, `POPPCT_URBAN`, and `is_hurricane`-and adds two engineered features: `log_population = log1p(POPULATION)` and `state_pop_bin`, which groups states into `small`, `medium`, and `large` population buckets based on their average census population. We tuned key capacity-controlling hyperparameters (`max_depth`, `learning_rate`, `max_leaf_nodes`, `min_samples_leaf`) using 5-fold cross-validation with a custom RMSE scorer defined on the original `CUSTOMERS.AFFECTED` scale, and selected the configuration that minimized cross-validated RMSE on the training data.

```
from sklearn.base import BaseEstimator, RegressorMixin, clone
from sklearn.metrics import mean_squared_error, make_scorer
import numpy as np
import pandas as pd

class LogTargetRegressor(BaseEstimator, RegressorMixin):
    """
    Wrap a base regressor so that it:
      - fits on log1p(y)
      - predicts on log scale
    Caller is responsible for applying expm1 to predictions
    when evaluating in original units.
    """
    def __init__(self, base_estimator=None):
        self.base_estimator = base_estimator

    def fit(self, X, y):
        y_log = np.log1p(y)
        self.reg_ = clone(self.base_estimator)
        self.reg_.fit(X, y_log)
        return self

    def predict(self, X):
        # returns predictions on log scale
        return self.reg_.predict(X)

    def get_params(self, deep=True):
        # so GridSearchCV can see through to base_estimator params
        params = {"base_estimator": self.base_estimator}
        if deep and hasattr(self.base_estimator, "get_params"):
            for k, v in self.base_estimator.get_params(deep=True).items():
                params[f"base_estimator__{k}"] = v
        return params

    def set_params(self, **params):
        base_params = {}
```

```
        other_params = {}
        for k, v in params.items():
            if k.startswith("base_estimator__"):
                base_params[k.split("__", 1)[1]] = v
            else:
                other_params[k] = v

        if "base_estimator" in other_params:
            self.base_estimator = other_params["base_estimator"]

        if base_params:
            if self.base_estimator is None:
                raise ValueError("base_estimator must be set before setting its
    params")
            self.base_estimator.set_params(**base_params)
        return self


def rmse_original(y_true, y_pred_log):
    """
    y_pred_log is on log scale; convert back and compute RMSE in customers.
    """
    y_pred = np.expm1(y_pred_log)
    y_pred = np.maximum(y_pred, 0.0)
    return np.sqrt(mean_squared_error(y_true, y_pred))


neg_rmse_original_scorer = make_scorer(
    rmse_original,
    greater_is_better=False
)
```

```
print(model_df['state_pop_bin'].value_counts())
```

```
state_pop_bin
large     595
medium    362
small      88
Name: count, dtype: int64
```

```
# Start from model_df already defined as in your Step 6
model_df = df.dropna(subset=['CUSTOMERS.AFFECTED']).copy()

nominal_features = ['U.S._STATE', 'NERC.REGION', 'CLIMATE.CATEGORY',
    'CAUSE.CATEGORY']
quantitative_features = ['ANOMALY.LEVEL', 'POPULATION', 'POPPCT_URBAN']
binary_features = ['is_hurricane']

# ---- New engineered features ----

# 1) log_population: compresses extreme population differences
model_df['log_population'] = np.log1p(model_df['POPULATION'])
```

```python
# 2) state_pop_bin: small / medium / large based on state-level mean POPULATION
state_mean_pop = model_df.groupby('U.S._STATE')['POPULATION'].mean()

bins = [0, 2e6, 8e6, np.inf]       # -02M, -28M, >8M
labels = ['small', 'medium', 'large']

state_pop_bin_map = pd.cut(
    state_mean_pop,
    bins=bins,
    labels=labels,
    include_lowest=True
)

model_df['state_pop_bin'] = model_df['U.S._STATE'].map(state_pop_bin_map)

# Drop rows missing the new feature (should be rare)
model_df = model_df.dropna(subset=['state_pop_bin'])

# Updated feature lists
nominal_features_extended = nominal_features + ['state_pop_bin']
quantitative_features_extended = quantitative_features + ['log_population']
binary_features_extended = binary_features  # unchanged

all_features_final = (
    nominal_features_extended
    + quantitative_features_extended
    + binary_features_extended
)
target = 'CUSTOMERS.AFFECTED'

# Drop rows with any NaNs in final features
model_df = model_df.dropna(subset=all_features_final + [target])

# Train/test split (80/20)
X = model_df[all_features_final]
y = model_df[target]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Train rows: {len(X_train)}, Test rows: {len(X_test)}")
print("Train bins:\n", X_train['state_pop_bin'].value_counts())
print("Test bins:\n", X_test['state_pop_bin'].value_counts())
```

```
Train rows: 836, Test rows: 209
Train bins:
 state_pop_bin
large     477
medium    289
small      70
Name: count, dtype: int64
Test bins:
```

```
 state_pop_bin
large     118
medium     73
small      18
Name: count, dtype: int64
```

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV, KFold

# Preprocessor with engineered features
preprocessor_final = ColumnTransformer(
    transformers=[
        ('nominal', OneHotEncoder(handle_unknown='ignore', sparse_output=False),
         nominal_features_extended),
        ('quantitative', StandardScaler(), quantitative_features_extended),
        ('binary', 'passthrough', binary_features_extended)
    ]
)

# Base gradient boosting model (on log target via LogTargetRegressor)
hgb = HistGradientBoostingRegressor(
    loss='squared_error',
    max_depth=None,
    random_state=42
)

log_hgb = LogTargetRegressor(base_estimator=hgb)

final_pipeline = Pipeline([
    ('preprocessor', preprocessor_final),
    ('model', log_hgb)
])

# Hyperparameter grid (only key capacity/regularization parameters)
param_grid = {
    'model__base_estimator__max_depth': [3, 5, None],
    'model__base_estimator__learning_rate': [0.05, 0.1, 0.2],
    'model__base_estimator__max_leaf_nodes': [15, 31, 63],
    'model__base_estimator__min_samples_leaf': [20, 50, 100],
}

cv = KFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(
    estimator=final_pipeline,
    param_grid=param_grid,
    scoring=neg_rmse_original_scorer,
    cv=cv,
    n_jobs=-1,
    verbose=1
)
```

```
grid.fit(X_train, y_train)

print("Best params:", grid.best_params_)
print("Best CV RMSE (orig):", -grid.best_score_)
```

```
Fitting 5 folds for each of 81 candidates, totalling 405 fits
Best params: {'model__base_estimator__learning_rate': 0.05,
    'model__base_estimator__max_depth': None,
    'model__base_estimator__max_leaf_nodes': 15,
    'model__base_estimator__min_samples_leaf': 20}
Best CV RMSE (orig): 290297.2329384634
```

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

# 1. Refit final model with best params on full training data
best_final_model = grid.best_estimator_
best_final_model.fit(X_train, y_train)

# 2. Predict on test set (log scale from LogTargetRegressor)
y_test_pred_log = best_final_model.predict(X_test)

# 3. Convert back to original scale
y_test_pred_final = np.expm1(y_test_pred_log)
y_test_pred_final = np.maximum(y_test_pred_final, 0.0)

# 4. Evaluate Final Model on test set
rmse_final = np.sqrt(mean_squared_error(y_test, y_test_pred_final))
r2_final = r2_score(y_test, y_test_pred_final)

print("=" * 55)
print("Final Model: HGB + Engineered Features (Log-Target)")
print("=" * 55)
print(f"Test RMSE:  {rmse_final:>12,.0f} customers")
print(f"Test R²:    {r2_final:>12.4f}")
print(f"Test mean:  {y_test.mean():>12,.0f} customers")
print(f"Test std:   {y_test.std():>12,.0f} customers")
print("=" * 55)

# 5. Compare to baseline (from Step 6)
abs_improvement = rmse_baseline - rmse_final
rel_improvement = abs_improvement / rmse_baseline * 100

print("\nPerformance Comparison vs Baseline")
print("-" * 40)
print(f"Baseline RMSE: {rmse_baseline:,.0f} customers")
print(f"Final RMSE:    {rmse_final:,.0f} customers")
print(f"Absolute ΔRMSE: {abs_improvement:,.0f} customers")
print(f"Relative ΔRMSE: {rel_improvement:,.1f}%")
print(f"Baseline R²: {r2_baseline:.4f}")
print(f"Final R²:    {r2_final:.4f}")
if rmse_final < rmse_baseline:
    print("\nFINAL MODEL is BETTER than baseline (lower RMSE).")
```

```
else:
    print("\nFINAL MODEL is NOT better than baseline (RMSE did not improve).")
```

```
=======================================================
Final Model: HGB + Engineered Features (Log-Target)
=======================================================
Test RMSE:       213,321 customers
Test R²:          0.0343
Test mean:       131,051 customers
Test std:        217,593 customers
=======================================================

Performance Comparison vs Baseline
----------------------------------------
Baseline RMSE: 223,420 customers
Final RMSE:    213,321 customers
Absolute ΔRMSE: 10,099 customers
Relative ΔRMSE: 4.5%
Baseline R²: -0.0594
Final R²:    0.0343

FINAL MODEL is BETTER than baseline (lower RMSE).
```

1.7.2 Final Model Plots

To inspect how the final model behaves on the held-out test set, we replicated the baseline diagnostic plots using the gradient-boosting predictions. The **Predicted vs Actual** scatter plot shows each outage as a point with the x-axis equal to the true `CUSTOMERS.AFFECTED` and the y-axis equal to the model's prediction, along with a dashed 45-degree reference line indicating perfect accuracy. Compared to the baseline plot, the final model's points cluster closer to this diagonal, especially for medium-sized outages, indicating that the gradient-boosting model has reduced systematic under- and over-prediction. The **Residual vs Predicted** plot, which graphs `(Actual - Predicted)` against the predicted value, shows a tighter, more symmetric band of residuals around zero than the Ridge baseline, suggesting better calibration across the range of predicted outage sizes and fewer extreme errors for typical events.

```
# Final Model Diagnostics: Predicted vs Actual and Residual Plot

# Assumes you have already computed:
# - y_test (true CUSTOMERS.AFFECTED on test set)
# - y_test_pred_final (final model predictions in original units)

import pandas as pd
import plotly.express as px
import numpy as np

# 1. Predicted vs Actual scatter plot
diag_final_df = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_test_pred_final
})
```

```
fig_final_pred = px.scatter(
    diag_final_df,
    x='Actual',
    y='Predicted',
    title='Final Model: Predicted vs Actual (CUSTOMERS.AFFECTED)',
    labels={'Actual': 'Actual Customers Affected',
            'Predicted': 'Predicted Customers Affected'},
    opacity=0.5
)

max_val_final = max(diag_final_df['Actual'].max(), diag_final_df['Predicted'].max())
fig_final_pred.add_shape(
    type='line',
    x0=0,
    y0=0,
    x1=max_val_final,
    y1=max_val_final,
    line=dict(dash='dash', color='red', width=1)
)

fig_final_pred.write_html(
    '../data/docs/img/plots/step7.1_final_pred_vs_actual.html',
    include_plotlyjs='cdn'
)
fig_final_pred.show()

# 2. Residual (Actual - Predicted) vs Predicted plot
residuals_final = y_test.values - y_test_pred_final

fig_final_resid = px.scatter(
    x=y_test_pred_final,
    y=residuals_final,
    title='Final Model: Residual Plot',
    labels={'x': 'Predicted Customers Affected',
            'y': 'Residual (Actual - Predicted)'},
    opacity=0.5
)

fig_final_resid.add_hline(
    y=0,
    line_dash='dash',
    line_color='red'
)

fig_final_resid.write_html(
    '../data/docs/img/plots/step7.2_final_residuals.html',
    include_plotlyjs='cdn'
)
fig_final_resid.show()
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.7.3 Final Model Results

On the held-out test set, the baseline Ridge regression with a log-transformed target achieved an RMSE of **223,420** customers and an R^2 of **-0.0594**, with RMSE slightly exceeding the test-set standard deviation of `CUSTOMERS.AFFECTED` (217,593 customers). In contrast, the final Histogram-based Gradient Boosting model with `log_population` and `state_pop_bin` achieved an RMSE of **213,321** customers and an $R^{2} of **0.0343** on the same test data. This corresponds to an absolute improvement of **10,099** customers and a **4.5%** reduction in RMSE, along with a shift from negative to slightly positive R^2. In practical terms, the final model reduces prediction error relative to the linear baseline, though the overall prediction task remains difficult because outage severity is highly skewed and noisy.

1.8 Step 8: Fairness Analysis

1.8.0.1 Fairness Setup: Groups and Metric

For our fairness analysis, we examine whether the final model is equally accurate for outages in large-population states versus smaller states. Using the engineered feature `state_pop_bin`, we define **Group A** as outages occurring in `large` states (average population above 8 million) and **Group B** as outages occurring in `small` and `medium` states combined (average population between about 0.6 and 8 million). Our fairness metric is the group-wise Root Mean Squared Error (RMSE) of the final model on the held-out test set, computed separately for Group A and Group B in the original `CUSTOMERS.AFFECTED` units.

We formalize the fairness question as a hypothesis test:

- **Null hypothesis** (H_0): The model's population-level RMSE is the same for large and smaller states, (RMSE_A = RMSE_B).

- **Alternative hypothesis** (H_1): The model's RMSE differs between the groups, (RMSE_A RMSE_B).

We fix the significance level at (= 0.05) and use a two-sided permutation test on the test set to assess whether the observed difference in RMSE could plausibly arise if the model were equally accurate across `state_pop_bin` groups.

```
from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

# Build a test DataFrame with predictions and group labels
test_df = X_test.copy()
test_df = test_df.assign(
    y_true=y_test.values,
    y_pred=y_test_pred_final,
    state_pop_bin=test_df['state_pop_bin'].astype(str)
)
```

```
# Define groups
group_A_mask = test_df['state_pop_bin'] == 'large'
group_B_mask = test_df['state_pop_bin'].isin(['small', 'medium'])

def group_rmse(df, mask):
    return np.sqrt(mean_squared_error(df.loc[mask, 'y_true'],
                                      df.loc[mask, 'y_pred']))

rmse_A = group_rmse(test_df, group_A_mask)
rmse_B = group_rmse(test_df, group_B_mask)

obs_diff = rmse_A - rmse_B  # A - B
print(f"Group A (large) RMSE:        {rmse_A:,.0f} customers")
print(f"Group B (small+medium) RMSE: {rmse_B:,.0f} customers")
print(f"Observed difference (A - B): {obs_diff:,.0f} customers")

# Permutation test
rng = np.random.default_rng(42)
n_permutations = 5000
perm_diffs = np.empty(n_permutations)

state_bins = test_df['state_pop_bin'].values

for i in range(n_permutations):
    # Permute group labels
    permuted = state_bins.copy()
    rng.shuffle(permuted)

    # Recompute group masks under permuted labels
    mask_A_perm = permuted == 'large'
    mask_B_perm = np.isin(permuted, ['small', 'medium'])

    # Compute RMSEs with permuted group assignments
    rmse_A_perm = group_rmse(test_df.assign(state_pop_bin=permuted), mask_A_perm)
    rmse_B_perm = group_rmse(test_df.assign(state_pop_bin=permuted), mask_B_perm)

    perm_diffs[i] = rmse_A_perm - rmse_B_perm

# Two-sided p-value
p_value = np.mean(np.abs(perm_diffs) >= np.abs(obs_diff))
print(f"Two-sided permutation p-value: {p_value:.4f}")
# Plot permutation distribution
# Plot permutation distribution of RMSE differences

import plotly.express as px
import pandas as pd

perm_df = pd.DataFrame({"perm_diff": perm_diffs})

fig_perm = px.histogram(
    perm_df,
    x="perm_diff",
    nbins=40,
```

```
    title="Permutation Distribution of RMSE Difference (large - small+medium)",
    labels={"perm_diff": "RMSE difference (A - B) in customers"},
    opacity=0.7
)

# Add vertical line for observed difference
fig_perm.add_vline(
    x=obs_diff,
    line_color="red",
    line_width=2,
    line_dash="dash",
    annotation_text=f"Observed diff = {obs_diff:,.0f}",
    annotation_position="top left"
)

fig_perm.update_layout(
    bargap=0.05,
    showlegend=False
)

fig_perm.write_html(
    "../data/docs/img/plots/step8.1_permutation_rmse_diff.html",
    include_plotlyjs="cdn"
)
fig_perm.show()
```

```
Group A (large) RMSE:        259,518 customers
Group B (small+medium) RMSE: 131,077 customers
Observed difference (A - B): 128,442 customers
Two-sided permutation p-value: 0.8226
```

```
Unable to display output for mime type(s): application/vnd.plotly.v1+json
```

1.8.0.2 Fairness Test: Permutation Procedure

To evaluate whether our predictive model exhibits bias based on state size, we conducted a formal fairness test. Specifically, we sought to determine if the model's accuracy-measured by **Root Mean Square Error (RMSE)** is significantly different for high-population states compared to smaller ones.

1.8.0.2.1 1. Group Definitions and Test Statistic

We partitioned the held-out test set into two distinct cohorts based on the `state_pop_bin` variable:

- **Group A (Large States):** Defined as `state_pop_bin = "large"`
- **Group B (Small/Medium States):** Defined as `state_pop_bin` $\in$ `["small", "medium"]`

To quantify the performance gap, we calculated the observed test statistic, T_{obs}, which represents the simple difference in RMSE between the two groups:

$$T_{\text{obs}} = \text{RMSE}_{\text{large}} - \text{RMSE}_{\text{small+medium}}$$

1.8.0.2.2 2. The Null Hypothesis and Permutation Logic

The **Null Hypothesis (H_0)** posits that the model is "fair" with respect to population size; that is, the model's error distribution is independent of the `state_pop_bin` label. If this is true, the specific labels (`"large"` vs. `"small/medium"`) are **exchangeable**. Any observed difference in RMSE would merely be a result of the specific random sample of outages in our test set rather than a systematic flaw in the model.

The **Alternative Hypothesis (H_1)** is that the model's RMSE differs between large and smaller states, i.e., $\text{RMSE}_{\text{large}} \neq \text{RMSE}_{\text{small+medium}}$.

1.8.0.2.3 3. The Permutation Procedure

To approximate the distribution of T under the null hypothesis, we employed a non-parametric permutation approach with **5,000 iterations**:

1.8.0.2.4 1. Label Shuffling: In each iteration b, we randomly redistribute the "large" and "small/medium" labels across the existing test-set observations. This preserves the joint distribution of predictions and actual values but breaks any real-world link between a state's size and its error.

1.8.0.2.5 2. Metric Recomputation: We calculate the RMSE for these new, randomized groups, $\textbf{RMSE}_A^{(b)}$ and $\textbf{RMSE}_B^{(b)}$.

1.8.0.2.6 3. Difference Mapping: We record the resulting difference $T^{(b)} = \textbf{RMSE}_A^{(b)} - \textbf{RMSE}_B^{(b)}$.

1.8.0.2.7 4. P-Value Calculation: We determine the two-sided p-value as the proportion of permuted differences $|T^{(b)}|$ that are equal to or greater than the magnitude of our original observed difference $|T_{\textbf{obs}}|$.

This procedure keeps the trained model and its predictions fixed and operates only on the test-set group labels, so it directly assesses whether the observed RMSE gap could be explained by random variation in group membership.

1.8.0.2.8 4. Findings and Statistical Conclusion

The empirical results from our test set are summarized below:

Group	RMSE (Customers)
Group A (Large)	259,518
Group B (Small + Medium)	131,077
Observed Difference (T_{obs})	**128,442**

Upon running the 5,000 permutations, we found that the resulting distribution of differences was relatively broad. Our observed difference of 128,442 customers yielded a **two-sided p-value of 0.8226**.

Conclusion: The two-sided permutation p-value is 0.8226, which is much larger than ($\alpha = 0.05$), we fail to reject the null hypothesis. In this test set, we do not find statistically significant evidence that RMSE differs between outages in large states and outages in small/medium states. This means the observed RMSE gap is plausible under the null model used in the permutation test, not that the model is definitively fair in every sense.

1.8.1 Project Summary

- This project analyzes major U.S. power outage events from 2000 to 2016 using a cleaned version of the Purdue LASCI outage dataset, beginning with 1,534 raw records and ending with 1,456 cleaned observations after removing duplicates and addressing structural data issues. The cleaning process standardized datetime fields, removed redundant columns, handled several targeted missingness problems, created an `is_hurricane` indicator, and preserved key geographic, climate, demographic, and outage-impact variables for later analysis.
- A major analytical question in the project was whether outage behavior differs meaningfully across locations beyond broad grid-region membership. To test this, the project focused on the WECC region and used a one-way ANOVA on log-transformed `OUTAGE.DURATION`, finding significant state-level variation with:

$$F \approx 5.0488 \quad \text{and} \quad p \approx 0.000001$$

  - This supports the idea that U.S.STATE contains useful predictive information beyond NERC region alone.

- The predictive task was then framed as a regression problem with response variable `CUSTOMERS.AFFECTED`, chosen as a direct operational measure of outage severity. Using features available at or shortly after outage onset, the project first built a Ridge regression baseline on a log-transformed target; this baseline achieved RMSE of 223,420 customers and $R^2 = -0.0594$, indicating weak predictive performance and motivating a more flexible model.

- The final model replaced Ridge with a Histogram-based Gradient Boosting Regressor trained on the same core predictors plus two engineered features, `log_population` and `state_pop_bin`. After 5-fold cross-validation over key hyperparameters, the final model achieved RMSE of 213,321 customers and $R^2 = 0.0343$ improving on the baseline by 10,099 customers, or about 4.5% in RMSE.
- The project ended with a fairness analysis comparing prediction accuracy across population-based state groups. Using a permutation test on held-out test-set RMSE, the project found Group A large-state RMSE of 259,518 customers, Group B small/medium-state RMSE of 131,077 customers, and a two-sided p-value of 0.8226, so it failed to find statistically significant evidence that model accuracy differed across those population tiers under the tested setup.

1.8.2 Closing Remarks

- This project developed an end-to-end analysis of major U.S. power outages, moving from data cleaning and missingness assessment to inference, prediction, and fairness evaluation. We found that outage duration varies significantly by state even within the same NERC region, which motivated the inclusion of location-based predictors in our modeling pipeline. We then framed outage severity prediction as a regression problem using `CUSTOMERS.AFFECTED` and showed that a Histogram-based Gradient Boosting model with engineered population features outperformed a Ridge regression baseline on held-out data. Finally, a permutation-based fairness analysis found no statistically significant evidence that model RMSE differed across state population tiers, suggesting that under this test-set definition the model's errors were not detectably uneven across those groups.